

Dokumentasi Teknis Arasvara

Dokumentasi arsitektur, infrastruktur, dan struktur data platform berita Arasvara

1. Arsitektur Sistem

1.1 Stack Teknologi

Core Framework & Runtime

- **Next.js 16.1.6** - React framework dengan App Router
- **React 19.2.3** - UI library dengan React Compiler enabled
- **TypeScript 5.9.3** - Type safety
- **Node.js** - Runtime environment

Database & Storage

- **MongoDB 7.1.0** - Primary database (NoSQL document store)
- **AWS S3** - Object storage untuk media (images, videos, files)
 - `@aws-sdk/client-s3 ^3.997.0`
 - `@aws-sdk/s3-request-presigner ^3.998.0`
- **IndexedDB** - Client-side storage untuk draft artikel dan pending media
 - `idb-keyval ^6.2.2`

Authentication & Security

- **Firebase Authentication 12.15.0** - User authentication provider
- **Firebase Admin 13.7.0** - Server-side Firebase operations
- **JWT** (`jsonwebtoken ^9.0.3`) - Token-based auth
- **bcryptjs ^3.0.3** - Password hashing

Media Processing

- **Sharp ^0.34.5** - Server-side image processing (resize, optimize, WebP conversion)
- **Pica ^9.0.1** - Client-side image resize (progressive, high quality)
- **fluent-ffmpeg ^2.1.3** - Video processing (thumbnail extraction)

Rich Text Editor

- **TipTap 3.15.3** - Headless WYSIWYG editor
 - Core + 10+ extensions (highlight, image, link, placeholder, text-align, underline, youtube, dll)

State Management & Data Fetching

- **@tanstack/react-query ^5.90.21** - Server state management
- **SWR ^2.4.1** - Data fetching hooks
- **React Hook Form ^7.71.2** - Form state management
- **Zod ^4.3.6** - Schema validation

UI Framework & Styling

- **Tailwind CSS 4.2.1** - Utility-first CSS framework
- **shadcn/ui ^4.8.0** - Component library berbasis Radix UI
- **Radix UI** - Headless UI primitives
- **Lucide React ^0.575.0** - Icon library
- **class-variance-authority ^0.7.1** - Component variants utility
- **tailwind-merge ^3.6.0** - Tailwind class merger

Animation & Interactions

- **GSAP ^3.14.2** - Animation library
- **@gsap/react ^2.1.2** - GSAP React bindings
- **Embla Carousel ^8.6.0** - Carousel library
- **Swiper ^12.1.3** - Touch slider
- **@dnd-kit** - Drag and drop library

Analytics & Monitoring

- **Google Analytics 4** - Web analytics
 - **@google-analytics/admin ^9.2.0**
- **Chart.js ^4.5.1** - Data visualization
- **react-chartjs-2 ^5.3.1** - Chart.js React wrapper
- **Pino ^10.3.1** - Structured logging
- **pino-pretty ^13.1.3** - Log formatter

UI Components & Utilities

- **cmdk ^1.1.1** - Command palette
- **Vaul ^1.1.2** - Drawer component
- **Sonner ^2.0.7** - Toast notifications
- **react-day-picker ^9.14.0** - Date picker
- **react-dropzone ^15.0.0** - File upload
- **react-image-crop ^11.1.2** - Image cropping
- **date-fns ^4.1.0** - Date utilities
- **slugify ^1.6.6** - String to slug converter
- **ulid ^3.0.2** - Unique ID generator

Social Media Embeds

- **react-social-media-embed ^2.5.18** - Embed social media posts
- **react-tweet ^3.3.0** - Twitter/X embed

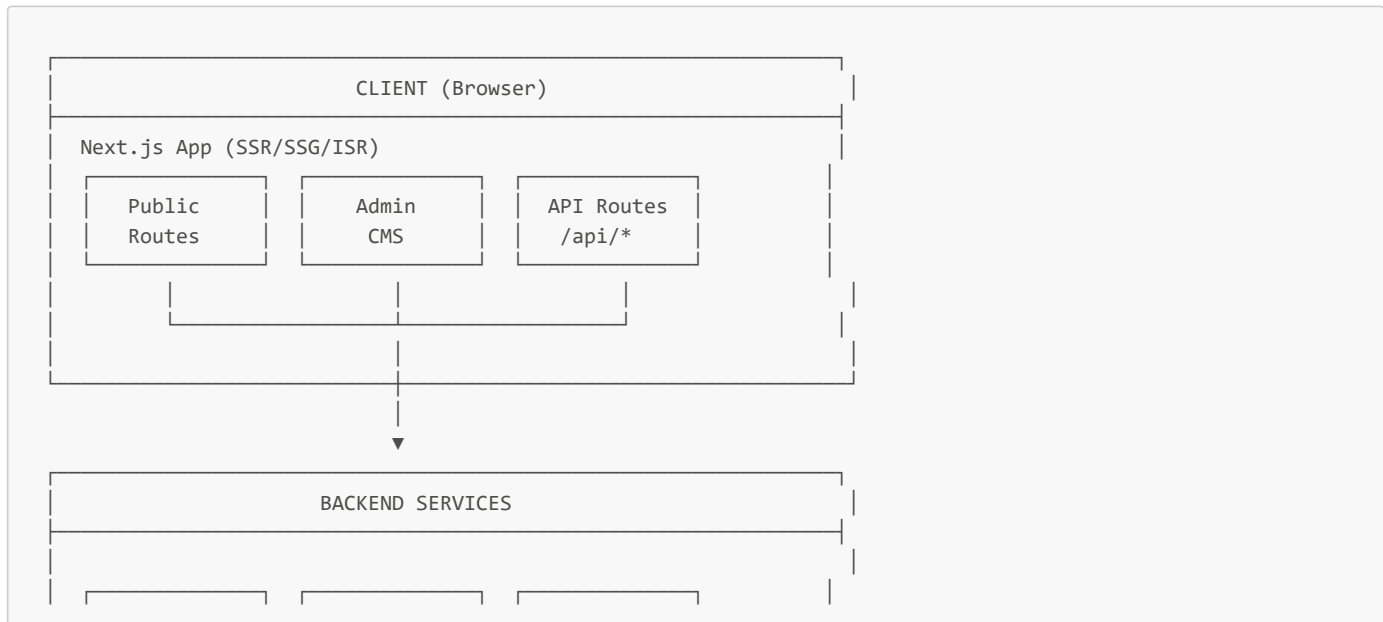
Testing

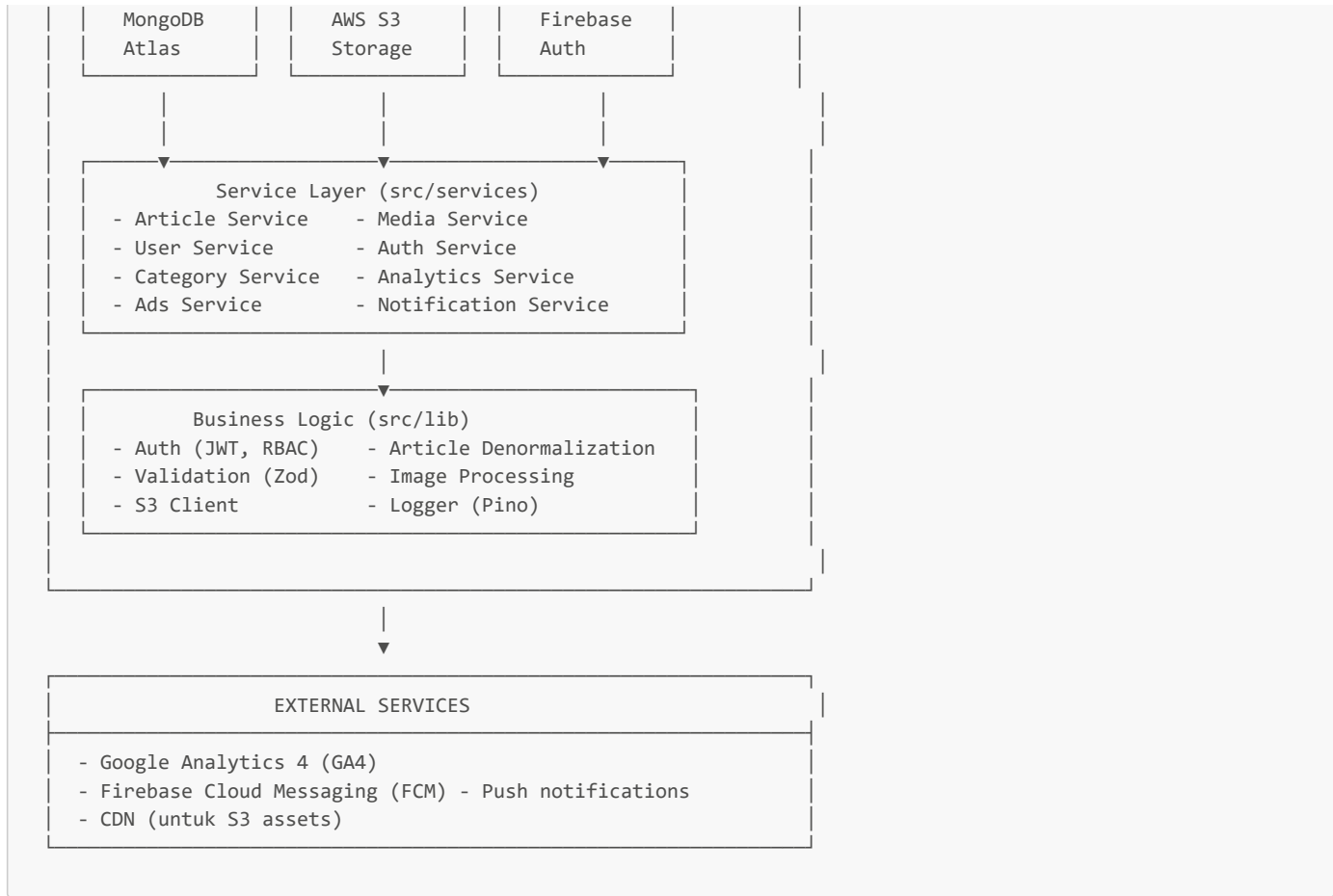
- **Vitest ^4.1.9** - Unit test framework

Build & Dev Tools

- **tsx ^4.21.0** - TypeScript execution untuk scripts
- **ESLint ^9** - Code linting
- **babel-plugin-react-compiler 1.0.0** - React Compiler (experimental)

1.2 Diagram Arsitektur High-Level





1.3 Infrastruktur & Deployment

Database: MongoDB Atlas

- **Cluster:** Shared/Dedicated (sesuai environment)
- **Database Name:** `arasvara_news` (configurable via `DB_NAME`)
- **Connection:** Via MongoDB driver dengan connection pooling
- **Collections:** 20+ collections (articles, users, categories, media, dll)
- **Indexes:** Compound indexes untuk performa query
- **Backup:** Automated snapshots (Atlas managed)

Storage: AWS S3

- **Buckets:**
 - Media bucket (images, videos)
 - Avatar bucket (user avatars)
 - Configuration bucket (site config files)
- **Access:** IAM credentials dengan least privilege
- **Upload:** Presigned URLs (client upload langsung ke S3)
- **CDN:** CloudFront atau S3 static hosting dengan cache headers
- **Immutability:** Cache-Control: immutable untuk uploaded assets

Authentication: Firebase

- **Provider:** Email/Password (extensible untuk Google, Facebook, dll)
- **Custom Claims:** Role injection via Firebase Admin SDK
- **Token:** Firebase ID token ditukar dengan JWT access token
- **Refresh:** Refresh token rotation strategy

Hosting & Deployment

- **Platform:** Vercel / Self-hosted Node.js server
- **Build:** `next build` (SSR + SSG + ISR)

- **Environments:** dev, staging, production
- **Environment Variables:** `.env.local` (dev), environment variables (prod)

1.4 External Services Integration

Google Analytics 4 (GA4)

- **Integration:** Google Tag Manager (GTM) + Measurement Protocol
- **Custom Events:** article_view, article_share, ad_click, search, dll
- **Custom Dimensions:** author, category, article_format
- **Server-side Tracking:** Measurement Protocol API untuk server events

Firestore Cloud Messaging (FCM)

- **Push Notifications:** Web push untuk breaking news, article updates
- **Topics:** Category-based subscriptions
- **Token Management:** FCM token registration dan rotation

Social Media Embeds

- **Twitter/X:** react-tweet component
- **Instagram:** react-social-media-embed
- **TikTok:** react-social-media-embed
- **YouTube:** TipTap YouTube extension

2. Struktur Kode

2.1 Organisasi Direktori

```

arasvara/
├── src/
│   ├── app/                                     # Next.js App Router
│   │   ├── (public)/                          # Public routes (no auth)
│   │   │   ├── page.tsx                      # Homepage
│   │   │   ├── layout.tsx                   # Public layout
│   │   │   ├── HomePageClient.tsx          # Client component
│   │   │   ├── [category]/[yyyy]/[mm]/[dd]/[slug]/ # Article detail (structured URL)
│   │   │   │   └── page.tsx
│   │   │   ├── category/[category]/        # Category listing page
│   │   │   │   ├── page.tsx
│   │   │   │   └── CategoryClient.tsx
│   │   │   ├── indeks/                     # News index (all articles)
│   │   │   │   ├── page.tsx
│   │   │   │   └── NewsIndeksClient.tsx
│   │   │   ├── news/[...segments]/         # Legacy & structured URL handler
│   │   │   │   └── page.tsx
│   │   │   ├── penulis/[slug]/            # Author profile page
│   │   │   │   ├── page.tsx
│   │   │   │   └── AuthorClient.tsx
│   │   │   ├── search/                    # Search results
│   │   │   │   ├── page.tsx
│   │   │   │   └── SearchClient.tsx
│   │   ├── (inside)/                       # Static info pages
│   │   │   ├── layout.tsx
│   │   │   ├── about-us/                  # Tentang kami
│   │   │   │   ├── page.tsx
│   │   │   │   ├── AboutUsClient.tsx
│   │   │   │   └── MouseBouncing.tsx
│   │   │   ├── disclaimer/                # Disclaimer
│   │   │   │   └── page.tsx
│   │   │   ├── pedoman-media-siber/      # Pedoman jurnalistik
│   │   │   │   └── page.tsx
│   │   ├── admin/                         # Admin CMS (auth required)
│   │   │   ├── (dashboard)/

```

5 / 90

└─ section/	
└─ analytics/	# Analytics components
└─ categories/	# Category components
└─ configuration/	# Configuration components
└─ dashboard/	# Dashboard components
└─ homepage/	# Homepage sections
└─ ┬─ carousel/	
└─ └─ socmed/	
└─ icon/	# Custom icons
└─ image-notfound/	# Fallback image component
└─ layout/	# Layout components
└─ media/	# Media components
└─ modal/	# Modal dialogs
└─ monthlyTarget/	# Monthly target components
└─ navbar/	# Navbar components
└─ navigation/	# Navigation components (Sidebar)
└─ news/	# Article display components
└─ notification/	# Notification components
└─ profile/	# Profile components
└─ search/	# Search components
└─ selected-topics/	# Selected topics components
└─ settings/	# Settings components
└─ sidebarPublic/	# Public sidebar
└─ social-link/	# Social link components
└─ sponsor/	# Sponsor components
└─ table/	# Data table components
└─ tags/	# Tag components
└─ teams/	# Team components
└─ tiptap/	# TipTap editor extensions & UI
└─ ui/	# shadcn/ui base components
└─ ┬─ button.tsx	
└─ ┬─ input.tsx	
└─ ┬─ dialog.tsx	
└─ ┬─ dropdown-menu.tsx	
└─ ┬─ select.tsx	
└─ ┬─ textarea.tsx	
└─ ┬─ card.tsx	
└─ ┬─ toast.tsx	
└─ └─ [30+ shadcn components]	
└─ users/	# User components
└─ hooks/	# Custom React hooks
└─ ┬─ animation/	# Animation hooks
└─ ┬─ carousel/	# Carousel hooks
└─ ┬─ use-ads.ts	
└─ ┬─ use-article.ts	
└─ ┬─ use-auth.ts	
└─ ┬─ use-category.ts	
└─ ┬─ use-debounce.ts	
└─ ┬─ use-media.ts	
└─ ┬─ use-notification.ts	
└─ └─ [other hooks]	
└─ lib/	# Core libraries & utilities
└─ ┬─ cache/	# Cache configuration
└─ └─ ┬─ article-isr.ts	
└─ ┬─ configuration/	# Configuration utils
└─ └─ ┬─ s3-config.ts	
└─ └─ └─ indexeddb-config.ts	
└─ ┬─ db/	# Database utilities
└─ └─ ┬─ db.ts	# MongoDB connection
└─ └─ ┬─ draftImageDb.ts	# IndexedDB for drafts
└─ └─ ┬─ s3.ts	# S3 client
└─ └─ └─ seed.ts	# Database seeding
└─ ┬─ image/	# Image processing utils
└─ ┬─ media/	# Media URL builders
└─ ┬─ migrations/	# Data migration helpers
└─ ┬─ s3/	# S3 utilities
└─ └─ server/	# Server-side API client

```

├─ tiptap/                                # TipTap utilities
├─ validations/                          # Zod schemas
│   └─ article.schema.ts
│   └─ user.schema.ts
│   └─ category.schema.ts
│   └─ [other schemas]
├─ auth.ts                              # Auth utilities (JWT, bcrypt)
├─ auth-client.ts                       # ROLES & PERMISSIONS definitions
├─ auth-config.ts                       # Auth constants
├─ auth-cookies.ts                     # Cookie utilities
├─ article-denorm.ts                   # Article denormalization logic
├─ api-error.ts                        # API error handling
├─ firebase-client-config.ts           # Firebase client config
├─ firebaseAdmin.ts                   # Firebase Admin SDK
├─ google-analytics.ts                # GA4 integration
├─ measurement-protocol.ts            # GA4 Measurement Protocol
├─ logger.ts                          # Pino logger
├─ utils.ts                           # General utilities
├─ services/                          # Business logic & external services (52 files)
│   └─ ads/                            # Ad services
│       └─ adClickService.ts
│       └─ AdsHomepageService.ts
│       └─ AdsSingleArticleService.ts
│       └─ AdsHistoryService.ts
│   └─ analytics/                     # Analytics services
│       └─ dashboard/
│           └─ audienceService.ts
│           └─ authorService.ts
│           └─ writerPerformanceService.ts
│       └─ editorActivityService.ts
│       └─ pushNotifService.ts
│       └─ viewArticleService.ts
│       └─ pageviewService.ts
│       └─ workflowService.ts
│   └─ article/                      # Article services
│       └─ articleSection/           # Article sections
│           └─ socmed/
│               └─ carouselService.ts
│               └─ featuredService.ts
│               └─ headlineService.ts
│               └─ popularService.ts
│               └─ editorChoiceService.ts
│       └─ getArticleService.ts
│       └─ writeArticleService.ts
│       └─ searchArticleService.ts
│       └─ relatedArticleService.ts
│   └─ reports/                      # Report services
│       └─ articleWriterService.ts
│       └─ kpiUserService.ts
│   └─ sponsor/
│       └─ SponsorService.ts
├─ auditLogService.ts
├─ authService.ts
├─ categoryService.ts
├─ configurationService.ts
├─ indeksService.ts
├─ mediaService.ts
├─ monthlyTargetService.ts
├─ notificationService.ts
├─ pushNotifService.ts
├─ refreshTokenService.ts
├─ registerUser.ts
├─ searchService.ts
├─ selectedTopicService.ts
├─ sitemapService.ts
├─ socialLinkService.ts
├─ tagsService.ts
├─ teamService.ts

```

```

└─ userService.ts

└─ types/                                # TypeScript type definitions
  └─ analytics/                          # Analytics types
  └─ reports/                            # Report types
  └─ aboutUs.ts
  └─ ads.ts
  └─ article.ts                          # Article types (Article, ArticleStatus, dll)
  └─ articleSection.ts
  └─ auditLog.ts
  └─ card.ts
  └─ category.ts
  └─ configuration.ts
  └─ general.ts
  └─ media.ts
  └─ monthlyTarget.ts
  └─ notification.ts
  └─ search.ts
  └─ selectedTopic.ts
  └─ sitemap.ts
  └─ sociallink.ts
  └─ sponsor.ts
  └─ team.ts
  └─ user.ts

└─ styles/                              # Global styles
  └─ globals.css

└─ proxy.ts                             # Proxy configuration

└─ scripts/                             # Maintenance & migration scripts
  └─ migrate-featured-image-filename.ts
  └─ migrate-webp-audit.ts
  └─ migrate-editor-activities-to-audit-log.ts
  └─ migrate-structured-path-remove-news-prefix.ts
  └─ audit-article-titles.ts
  └─ audit-article-paths.ts
  └─ audit-article-publish-dates.ts
  └─ audit-user-names-slugs.ts
  └─ backfill-article-public-path.ts
  └─ backfill-user-slugs.ts
  └─ upgrade-articles-to-structured-path.ts
  └─ verify-public-media-url.ts
  └─ verify-cdn-phases.ts
  └─ verify-performance-opts.ts
  └─ verify-editor-activities-migration.ts
  └─ warm-article-paths-cache.ts
  └─ register-ga-custom-definitions.ts
  └─ check-author-slug.ts

└─ documentation/                       # Documentation
  └─ tech-docs.md                       # This file

└─ memory/                             # Project memory (knowledge base)
  └─ role.md                            # Roles & permissions documentation
  └─ alurArticle.md                     # Article workflow documentation
  └─ analytics.md                       # Analytics documentation
  └─ KPI.md                             # KPI definitions
  └─ refactorMediaArticle.md            # Media refactoring notes
  └─ searchLogic.md                     # Search logic documentation

└─ public/                             # Static assets
  └─ images/
  └─ [other static files]

└─ .env.example                         # Environment variables template
└─ .env.local                           # Local environment (gitignored)
└─ package.json
└─ tsconfig.json

```



```
├─ tailwind.config.ts
├─ next.config.ts
├─ vitest.config.ts
├─ eslint.config.mjs
└─ README.md
```

2.2 Pola Arsitektur

Route Groups

- **(public)**: Route publik tanpa auth (homepage, article, category, search)
 - Layout: navbar + footer
 - ISR (Incremental Static Regeneration) untuk artikel
- **(inside)**: Halaman statis info (about-us, disclaimer, pedoman)
 - Layout: navbar + footer + custom styling
- **admin**: CMS admin panel (auth required)
 - Layout: sidebar + header
 - Middleware auth check
 - Role-based route protection

Server vs Client Components

- **Server Components** (default):
 - Layout components
 - Data fetching pages (article detail, category listing)
 - API data fetching via `fetch()` atau MongoDB direct access
 - SEO-critical content
- **Client Components** ('use client'):
 - Interactive forms (TipTap editor, search input)
 - State management (React Query, SWR)
 - Event handlers (onClick, onChange)
 - Animations (GSAP, Framer Motion)
 - Browser APIs (localStorage, IndexedDB)

Data Fetching Strategy

1. **Server-side** (Server Components):
 - MongoDB direct queries via `getCollection()`
 - No API call overhead
 - SEO-friendly
2. **Client-side** (Client Components):
 - React Query untuk server state caching
 - SWR untuk real-time data (dashboard, analytics)
 - API routes via `fetch('/api/...')`
3. **Hybrid** (ISR):
 - Article pages: SSG dengan revalidation (configurable TTL)
 - Category pages: SSG dengan on-demand revalidation

Error Handling Pattern

```
// API Routes
try {
  // ... business logic
  return NextResponse.json({ data }, { status: 200 });
} catch (error) {
  logger.error({ error }, 'Error context');
  return NextResponse.json(
    { error: 'User-friendly message' },
    { status: 500 }
  );
}
```

```
// Server Components
async function getData() {
  try {
    return await fetchData();
  } catch (error) {
    // Log & return fallback
    logger.error({ error });
    return { data: [], error: true };
  }
}

// Client Components
const { data, error, isLoading } = useQuery({
  queryKey: ['articles'],
  queryFn: fetchArticles,
  retry: 3,
  staleTime: 5 * 60 * 1000,
});

if (error) return <ErrorComponent />;
if (isLoading) return <Skeleton />;
```

Logging Strategy

- **Library:** Pino (structured JSON logging)
- **Levels:** trace, debug, info, warn, error, fatal
- **Context:** Setiap log include `requestId`, `userId`, `path`
- **Production:** `info` level ke stdout (captured by hosting platform)
- **Development:** `debug` level dengan pretty formatting

```
import { logger } from "@lib/logger";

logger.info({ articleId, userId }, "Article published");
logger.error({ error, context }, "Failed to upload media");
```

2.3 Konvensi Kode

Naming Conventions

- **Files:**
 - Components: PascalCase (`ArticleCard.tsx`)
 - Utilities: camelCase (`auth.ts`, `utils.ts`)
 - Types: camelCase dengan `.ts` extension (`article.ts`)
 - API routes: lowercase dengan hyphen (`article-view.ts`)
- **Variables:**
 - camelCase untuk variables & functions (`getUserById`)
 - PascalCase untuk React components (`ArticleEditor`)
 - UPPER_SNAKE_CASE untuk constants (`MAX_FILE_SIZE`)
- **Types:**
 - PascalCase untuk interfaces & types (`Article`, `UserProfile`)
 - Enum: PascalCase dengan UPPER_CASE values (`ArticleStatus.PUBLISHED`)

File Organization

- **Co-location:** Component + styles + tests di folder yang sama

```
ArticleCard/
├─ ArticleCard.tsx
├─ ArticleCard.test.ts
└─ index.ts (barrel export)
```

- **Barrel exports:** Setiap folder punya `index.ts` untuk re-export
- **Separation of concerns:**
 - UI components di `components/`
 - Business logic di `services/`
 - Utilities di `lib/`
 - Types di `types/`

Import Ordering

1. External packages (React, Next.js, dll)
2. Internal absolute imports (`@/components`, `@/lib`)
3. Relative imports (`./`, `../`)
4. Type imports (gunakan `import type`)
5. CSS imports (terakhir)

```
import { useState } from "react";
import { useRouter } from "next/navigation";

import { Button } from "@/components/ui/button";
import { getArticle } from "@/services/article/getArticleService";
import { logger } from "@/lib/logger";

import type { Article } from "@/types/article";

import "@/styles.css";
```

Code Splitting Strategy

- **Route-based:** Automatic via Next.js App Router
- **Component-based:** Dynamic imports untuk heavy components

```
const TipTapEditor = dynamic(() => import('@/components/typetap/Editor'), {
  ssr: false,
  loading: () => <EditorSkeleton />,
});
```

- **Library-based:** Tree-shaking via named imports

```
// Good: Tree-shakeable
import { format } from "date-fns";

// Bad: Bundles entire library
import dateFns from "date-fns";
```

3. Database Schema & Data Model

3.1 MongoDB Collections Overview

Database Name: `arasvara_news` (configurable via env `DB_NAME`)

Total Collections: 25+ collections

Kategori Collections:

1. **Core Content:** articles, categories, media, tags
2. **User Management:** users, teams, refresh_tokens
3. **Editorial:** audit_log, editor_activities, notifications

- 4. **Homepage:** section_articles, carousel_section, video_section, selected_topics
- 5. **Advertising:** ads_homepage, ads_article, ad_click_events, sponsors
- 6. **Analytics:** article_views, page_views, monthly_targets
- 7. **Configuration:** configuration, social_links
- 8. **Push Notification:** push_tokens, category_push_subscriptions

3.2 Collection: **articles**

Purpose: Menyimpan artikel berita (format STANDARD dan GALLERY)

Schema:

```
{
  _id: ObjectId,

  // Content
  title: string,           // Judul artikel
  slug: string,            // URL-friendly slug (unique)
  excerpt: string,        // Ringkasan artikel
  content?: string,       // HTML dari TipTap (untuk STANDARD format)
  format: "STANDARD" | "GALLERY", // Tipe artikel (immutable setelah dibuat)

  // Taxonomy
  categoryId: ObjectId,    // Reference ke categories._id
  category: Category,     // Denormalized category object
  tags: Tag[],            // Array of {_id, name, slug}

  // Media
  featuredImage?: {
    mediaId: ObjectId,    // Reference ke media._id
    url: string,          // CDN URL (built on read)
    caption: string,
    credit: string,
    media?: Media | null  // Populated media object
  },
  contentMedia?: ArticleMedia[], // Media di dalam content (STANDARD)
  galleryItems?: GalleryItem[],  // Gallery items dengan order (GALLERY)

  // Authorship & Attribution
  authorId: ObjectId,      // Reference ke users._id (primary author)
  author: UserProfile,     // Denormalized author
  editorId?: ObjectId | null, // Reference ke users._id (editor)
  editor?: UserProfile | null, // Denormalized editor
  contributorIds?: ObjectId[], // Array reference ke users._id
  contributors?: UserProfile[], // Denormalized contributors

  // Publishing
  status: ArticleStatus,  // DRAFT | PENDING_REVIEW | PUBLISHED | SCHEDULED | REJECTED |
TAKEN_DOWN | DELETED
  publishedAt: Date,      // Tanggal publish (atau scheduled date)
  publishedBy?: ObjectId, // User yang publish
  scheduledAt?: Date | null, // Scheduled publish date
  submittedAt?: Date,     // Tanggal submit ke review

  // SEO & URLs
  metaTitle?: string,    // Custom meta title
  metaDesc?: string,     // Custom meta description
  publicPath?: string | null, // Canonical path, e.g. /news/nasional/2026/06/19/judul
  urlFormat?: "legacy" | "structured", // URL format type

  // Homepage Flags
  isFeatured?: boolean,  // Featured section
  isHeadline?: boolean,  // Headline section
  isBreaking?: boolean,  // Breaking news badge
  isPopular?: boolean,   // Popular section
  isEditorChoices?: boolean, // Editor's choice section
```

```
// Analytics
viewCount: number, // Total views (default 0)

// History & Audit
revisionHistory?: ArticleRevision[], // Array of {by, at, from, to, reason}
contentUpdatedAt?: Date | null, // Last content edit after publish (untuk SEO)

// Timestamps
createdAt: Date,
createdBy?: ObjectId, // User yang create
updatedAt: Date,
deletedAt?: Date | null, // Soft delete timestamp

// Relations
relatedArticles?: SectionArticleItem[] // Related articles
}
```

Indexes:

```
{ slug: 1 } // unique
{ status: 1, publishedAt: -1 } // compound untuk listing
{ publicPath: 1 } // sparse, untuk structured URL lookup
{ categoryId: 1, publishedAt: -1 } // category listing
{ authorId: 1, publishedAt: -1 } // author profile
{ 'tags.slug': 1 } // tag search
{ status: 1, scheduledAt: 1 } // scheduled publish job
{ title: 'text', excerpt: 'text', content: 'text' } // text search
```

Relations:

- → **users** (via **authorId**, **editorId**, **contributorIds**, **createdBy**, **publishedBy**)
- → **categories** (via **categoryId**)
- → **media** (via **featuredImage.mediaId**, **contentMedia[].mediaId**, **galleryItems[].mediaId**)

Denormalization:

- **category**, **author**, **editor**, **contributors** disimpan denormalized untuk performa read
- Update via background job saat source data berubah

3.3 Collection: **users**

Purpose: User management (admin, editor, writer, subscriber)

Schema:

```
{
  _id: ObjectId,
  email: string, // Unique email
  password?: string, // bcrypt hashed (nullable untuk OAuth)
  name: string, // Display name
  slug?: string, // URL-friendly username untuk /penulis/[slug]
  nameNormalized?: string, // lowercase, no punctuation (untuk cek duplikat)
  role: "admin" | "editor-in-chief" | "managing-editor" | "head-of" |
    "editor" | "reporter" | "writer" | "contributor" |
    "account-executive" | "subscriber",
  teamId?: ObjectId, // Reference ke teams._id
  team?: Team, // Denormalized team info
  avatar?: string | AvatarUser, // Avatar URL atau object
  bio?: string, // Bio penulis
  isActive?: boolean, // Account active status (default true)
  createdAt: Date,
  updatedAt: Date,
```

```
deletedAt?: Date | null           // Soft delete
}
```

Indexes:

```
{
  email: 1;
} // unique
{
  slug: 1;
} // unique, sparse
{
  role: 1;
}
{
  nameNormalized: 1;
} // untuk cek duplikasi
{
  teamId: 1;
}
```

Relations:

- → `teams` (via `teamId`)
 - ← `articles` (via `authorId`, `editorId`, `contributorIds`)
-

3.4 Collection: `categories`

Purpose: Kategori artikel (hierarkis, nested)

Schema:

```
{
  _id: ObjectId,
  name: string,           // Nama kategori
  slug: string,           // URL-friendly (unique)
  nickname?: string,      // Nama pendek untuk display
  showOnNavbar?: boolean, // Tampilkan di navbar homepage
  description?: string,   // Deskripsi kategori
  order?: number,         // Display order (ascending)
  featured?: boolean,     // Featured category
  featuredOrder?: number, // Order di featured section
  parentId?: ObjectId | null, // Self-reference untuk nested categories
  createdAt?: Date,
  updatedAt?: Date
}
```

Indexes:

```
{ slug: 1 } // unique
{ parentId: 1 } // untuk tree query
{ showOnNavbar: 1, order: 1 } // navbar listing
{ featured: 1, featuredOrder: 1 } // featured listing
```

Relations:

- Self-reference via `parentId` (nested categories)
 - ← `articles` (via `categoryId`)
-

3.5 Collection: **media**

Purpose: Media assets (images, videos) yang diupload ke S3

Schema:

```
{
  _id: ObjectId,
  url: string,           // S3 URL atau CDN URL
  filename: string,      // Original filename
  mimetype: string,      // image/jpeg, image/webp, video/mp4, dll
  size: number,          // File size in bytes
  caption?: string,      // Media caption
  credit?: string,       // Photo/video credit
  watermark?: boolean,   // Apakah di-watermark
  createdAt: Date,
  updatedAt: Date
}
```

Indexes:

```
{
  filename: 1;
}
{
  createdAt: -1;
} // listing by date
```

Relations:

- ← **articles** (via **featuredImage.mediaId**, **contentMedia**, **galleryItems**)

Storage:

- File disimpan di AWS S3
- **url** field berisi S3 key atau full CDN URL
- Watermark diproses dengan Sharp sebelum upload

3.6 Collection: **teams**

Purpose: Tim redaksi / desk

Schema:

```
{
  _id: ObjectId,
  name: string,           // e.g. "Politik", "Ekonomi", "Olahraga"
  slug: string,           // URL-friendly
  description?: string,   // Deskripsi tim
  createdAt: Date,
  updatedAt: Date
}
```

Indexes:

```
{
  slug: 1;
} // unique
```

Relations:

- ← `users` (via `teamId`)

3.7 Collection: `audit_log`

Purpose: Audit trail untuk semua aksi editorial penting

Schema:

```
{
  _id: ObjectId,
  actor: {                                // User yang melakukan aksi
    _id: ObjectId,
    name: string,
    email: string,
    avatarUrl?: string
  },
  action: "CREATE" | "UPDATE" | "DELETE" | "PUBLISH" | "SCHEDULE" |
    "TAKE_DOWN" | "REJECT" | "RESTORE",
  entity: string,                        // "articles" | "section_featured" | "categories" | dll
  entityId: ObjectId,                  // ID dokumen yang diaudit
  details?: string,                    // Detail aksi
  oldValue?: any,                      // Snapshot before
  newValue?: any,                      // Snapshot after
  meta?: {                             // Contextual metadata
    statusFrom?: ArticleStatus,
    statusTo?: ArticleStatus,
    articleTitle?: string,
    sectionType?: string,
    platform?: string,
    reason?: string,
    articleCount?: number,
    originalId?: ObjectId,
    migratedFrom?: string
  },
  createdAt: Date,
  ipAddress?: string                  // IP address actor
}
```

Indexes:

```
{ entityId: 1, createdAt: -1 }
{ 'actor._id': 1, createdAt: -1 }
{ entity: 1, createdAt: -1 }
{ createdAt: -1 } // time-based queries
```

Relations:

- → `users` (via `actor._id`)
- → polymorphic via `entity` + `entityId`

3.8 Collection: `editor_activities`

Purpose: Aktivitas editorial untuk tracking KPI editor

Schema:

```
{
  _id: ObjectId,
  actor: {
    _id: ObjectId,
```



```

    name: string,
    email: string,
    avatarUrl?: string
  },
  action: "CREATE" | "UPDATE" | "DELETE" | "PUBLISH" | "SCHEDULE" |
    "TAKE_DOWN" | "REJECT" | "RESTORE",
  statusFrom?: ArticleStatus,
  statusTo?: ArticleStatus,
  reason?: string,
  article: {
    _id: ObjectId,
    title: string,
    author: {
      _id: ObjectId,
      name: string
    }
  },
  timestamp: Date,
  createdAt: Date,
  userId: ObjectId,           // Flat untuk agregasi
  articleId: ObjectId,       // Flat untuk agregasi
  authorId: ObjectId,        // Flat untuk agregasi
  meta?: {
    reason?: string
  },
  deletedAt?: Date | null
}

```

Indexes:

```

{ userId: 1, timestamp: -1 }
{ articleId: 1, timestamp: -1 }
{ timestamp: -1 }

```

3.9 Collection: notifications

Purpose: In-app notifications untuk user

Schema:

```

{
  _id: ObjectId,
  receiver: {                               // Target user
    _id: ObjectId,
    name: string,
    email: string,
    avatarUrl?: string
  },
  actor: {                                  // User yang trigger notifikasi
    _id: ObjectId,
    name: string,
    email: string,
    avatarUrl?: string
  },
  type: "article-submitted" | "article-published" | "article-revision" |
    "article-rejected" | "article-taken-down" | "article-deleted" |
    "article-raising" | "ads-raised" | "ads-taken-down" |
    "system-announcement" | "schedule-published" | "article-approval",
  title: string,                           // Notification title
  message: string,                         // Notification body
  targetId?: string,                       // ID dokumen terkait
  link?: string,                           // URL tujuan
  icon?: string,                           // Icon name
  imageUrl?: string,                       // Image URL
}

```

```
isPushSent?: boolean,           // Apakah sudah dikirim via FCM
readAt?: Date | null,          // Timestamp dibaca (null = unread)
createdAt: Date,
meta?: any                     // Additional metadata
}
```

Indexes:

```
{ 'receiver._id': 1, readAt: 1, createdAt: -1 }
{ 'receiver._id': 1, createdAt: -1 }
{ type: 1, createdAt: -1 }
```

Relations:

- → `users` (via `receiver._id`, `actor._id`)

3.10 Collection: `ads_homepage`**Purpose:** Iklan untuk homepage**Schema:**

```
{
  _id: ObjectId,
  name: string,           // Nama iklan
  position: "headline" | "tiktok" | "youtube" | "reels" | "popular" |
            "photography" | "above_photography" | "editor_choice" |
            "featured" | "horizontal_featured",
  span: 1 | 2,           // Grid span (1 atau 2 kolom)
  ratio?: "21:9" | "16:9" | "4:3", // Aspect ratio (untuk above_photography)
  banner: {              // Banner file
    url: string,
    filename: string,
    mimetype: string,
    size: number
  },
  linkUrl: string,        // Target URL
  order: number,          // Display order
  variant?: string,       // Variant type (optional)
  startedAt: Date,        // Start date
  endedAt: Date,          // End date
  isActive: boolean,      // Active status
  clicks: number,         // Click count
  createdAt: Date,
  updatedAt: Date,
  deletedAt?: Date | null
}
```

Indexes:

```
{ position: 1, isActive: 1, startedAt: 1, endedAt: 1 }
{ isActive: 1, startedAt: 1, endedAt: 1 }
{ endedAt: 1 } // untuk expired ads check
```

3.11 Collection: `ads_article`**Purpose:** Iklan untuk halaman artikel**Schema:**

```
{
  _id: ObjectId,
  name: string,
  categories: Array<{ // Target categories
    _id: ObjectId,
    slug: string
  }>,
  placement: "vertical" | "horizontal", // Placement type
  banner: {
    url: string,
    filename: string,
    mimetype: string,
    size: number
  },
  linkUrl: string,
  order: number,
  variant?: string,
  span: 1 | 2,
  startedAt: Date,
  endedAt: Date,
  isActive: boolean,
  clicks: number,
  createdAt: Date,
  updatedAt: Date,
  deletedAt?: Date | null
}
```

Indexes:

```
{ placement: 1, isActive: 1, startedAt: 1, endedAt: 1 }
{ 'categories._id': 1, isActive: 1 }
```

Relations:

- → `categories` (via `categories[]._id`)

3.12 Collection: `ad_click_events`

Purpose: Tracking klik iklan

Schema:

```
{
  _id: ObjectId,
  adId: ObjectId, // Reference ke ads_homepage atau ads_article
  adType: "homepage" | "article", // Tipe iklan
  clickedAt: Date,
  userId?: ObjectId, // User yang klik (optional)
  ipAddress?: string,
  userAgent?: string,
  referrer?: string
}
```

Indexes:

```
{ adId: 1, clickedAt: -1 }
{ clickedAt: -1 } // untuk aggregation & cleanup
```

3.13 Collection: `sponsors`

Purpose: Sponsor display

Schema:

```
{
  _id: ObjectId,
  name: string,
  image_url: string,           // Logo URL
  order: number,
  createdAt: Date,
  createdBy: ObjectId
}
```

Indexes:

```
{
  order: 1;
}
```

3.14 Collection: `article_views`

Purpose: Tracking view artikel

Schema:

```
{
  _id: ObjectId,
  articleId: ObjectId,
  userId?: ObjectId,           // User yang view (optional)
  sessionId?: string,          // Session ID
  ip?: string,
  userAgent?: string,
  referrer?: string,
  viewedAt: Date,
  deletedAt?: Date | null
}
```

Indexes:

```
{ articleId: 1, viewedAt: -1 }
{ viewedAt: -1 } // untuk aggregation & retention policy
```

Relations:

- → `articles` (via `articleId`)
 - → `users` (via `userId`)
-

3.15 Collection: `page_views`

Purpose: Tracking pageview seluruh site

Schema:

```
{
  _id: ObjectId,
```

```
path: string, // URL path
sessionId: string,
ip: string,
userAgent: string,
referrer?: string,
viewedAt: Date
}
```

Indexes:

```
{ path: 1, viewedAt: -1 }
{ viewedAt: -1 }
{ sessionId: 1 }
```

3.16 Collection: `refresh_tokens`

Purpose: JWT refresh token untuk auth

Schema:

```
{
  _id: ObjectId,
  userId: ObjectId,
  token: string, // Hashed token
  expiresAt: Date,
  createdAt: Date,
  revokedAt?: Date | null,
  replacedBy?: ObjectId | null // Token rotation tracking
}
```

Indexes:

```
{ userId: 1, expiresAt: -1 }
{ token: 1 } // untuk lookup
{ expiresAt: 1 } // TTL index untuk auto-cleanup
```

Relations:

- → `users` (via `userId`)

3.17 Collection: `monthly_targets`

Purpose: Target KPI bulanan (global atau per channel)

Schema:

```
{
  _id: ObjectId,
  key: "ARTICLES_SUBMITTED" | "ARTICLES_PUBLISHED" | "SOCIAL_MEDIA_PUBLISHED" |
      "ARTICLES_TO_PROCESS" | "REVISION_RATE_MAX" | "PROCESSING_TIME_SLA_MINUTES" |
      "SITE_TOTAL_PAGEVIEWS" | "CHANNEL_PAGEVIEWS" | "CHANNEL_ARTICLES" |
      "AD_CLICKS_MIN",
  value: number, // Target value
  period: string, // Format "YYYY-MM"
  scopeType: "GLOBAL" | "CHANNEL", // Scope type
  category?: { // Hanya untuk CHANNEL scope
    _id: ObjectId,
    name: string,
    slug: string
  }
}
```

```
  },
  createdAt: Date,
  updatedAt: Date
}
```

Indexes:

```
{ key: 1, period: 1, scopeType: 1 }
{ period: -1 }
{ 'category._id': 1, period: 1 }
```

Relations:

- → `categories` (via `category._id` untuk CHANNEL scope)
-

3.18 Collection: `selected_topics`

Purpose: Topik pilihan untuk homepage

Schema:

```
{
  _id: ObjectId,
  title: string,
  articles: ObjectId[],           // Array of article IDs
  order: number,
  isActive: boolean,
  createdAt: Date,
  updatedAt: Date
}
```

Indexes:

```
{ order: 1, isActive: 1 }
{ isActive: 1 }
```

Relations:

- → `articles` (via `articles[]`)
-

3.19 Collection: `section_articles`

Purpose: Artikel di section homepage (featured, headline, popular, editor choice)

Schema:

```
{
  _id: ObjectId,
  article_id: ObjectId,
  article: ArticleListResponse,   // Denormalized article
  order: number,
  type: "featured" | "editor choices" | "popular" | "headline",
  createdAt: Date,
  createdBy: ObjectId
}
```

Indexes:

```
{ type: 1, order: 1 }
{ article_id: 1 }
```

Relations:

- → `articles` (via `article_id`)
 - → `users` (via `createdBy`)
-

3.20 Collection: `carousel_section`

Purpose: Carousel homepage

Schema:

```
{
  _id: ObjectId,
  article_id: ObjectId,
  article: ArticleListResponse,      // Denormalized
  order: number,
  createdAt: Date,
  createdBy: ObjectId
}
```

Indexes:

```
{
  order: 1;
}
{
  article_id: 1;
}
```

3.21 Collection: `video_section`

Purpose: Video socmed section (TikTok, Instagram, YouTube)

Schema:

```
{
  _id: ObjectId,
  video_url: string,                // Embed URL
  title: string,
  thumbnail_url: string,
  thumbnail?: Media,               // Populated thumbnail object
  order: number,
  type: "tiktok" | "instagram" | "youtube",
  createdAt: Date,
  createdBy: ObjectId
}
```

Indexes:

```
{ type: 1, order: 1 }
```

3.22 Collection: `push_tokens`

Purpose: FCM push token registration

Schema:

```
{
  _id: ObjectId,
  userId: ObjectId,
  token: string,           // FCM token
  platform: "web" | "android" | "ios" | "other",
  createdAt: Date,
  updatedAt: Date
}
```

Indexes:

```
{
  userId: 1;
}
{
  token: 1;
} // unique
```

Relations:

- → `users` (via `userId`)
-

3.23 Collection: `category_push_subscriptions`

Purpose: User subscription ke push notification per category

Schema:

```
{
  _id: ObjectId,
  userId: ObjectId,
  categoryId: ObjectId,
  createdAt: Date
}
```

Indexes:

```
{ userId: 1, categoryId: 1 } // compound unique
{ categoryId: 1 }
```

Relations:

- → `users` (via `userId`)
 - → `categories` (via `categoryId`)
-

3.24 Collection: `tag_recommendations`

Purpose: Cache tag populer untuk recommendations

Schema:

```
{
  _id: ObjectId,
  name: string,
```



```
slug: string,
count: number,           // Usage count
createdAt: Date,
updatedAt: Date
}
```

Indexes:

```
{
  slug: 1;
} // unique
{
  count: -1;
} // ranking by usage
```

3.25 Collection: `configuration`

Purpose: Site configuration key-value store

Schema:

```
{
  _id: ObjectId,
  key: string,           // Config key (unique)
  value: string | boolean | object, // Config value
  type: "string" | "number" | "boolean" | "file",
  createdAt: Date,
  updatedAt: Date
}
```

Indexes:

```
{
  key: 1;
} // unique
```

3.26 Collection: `social_links`

Purpose: Social media links untuk footer/header

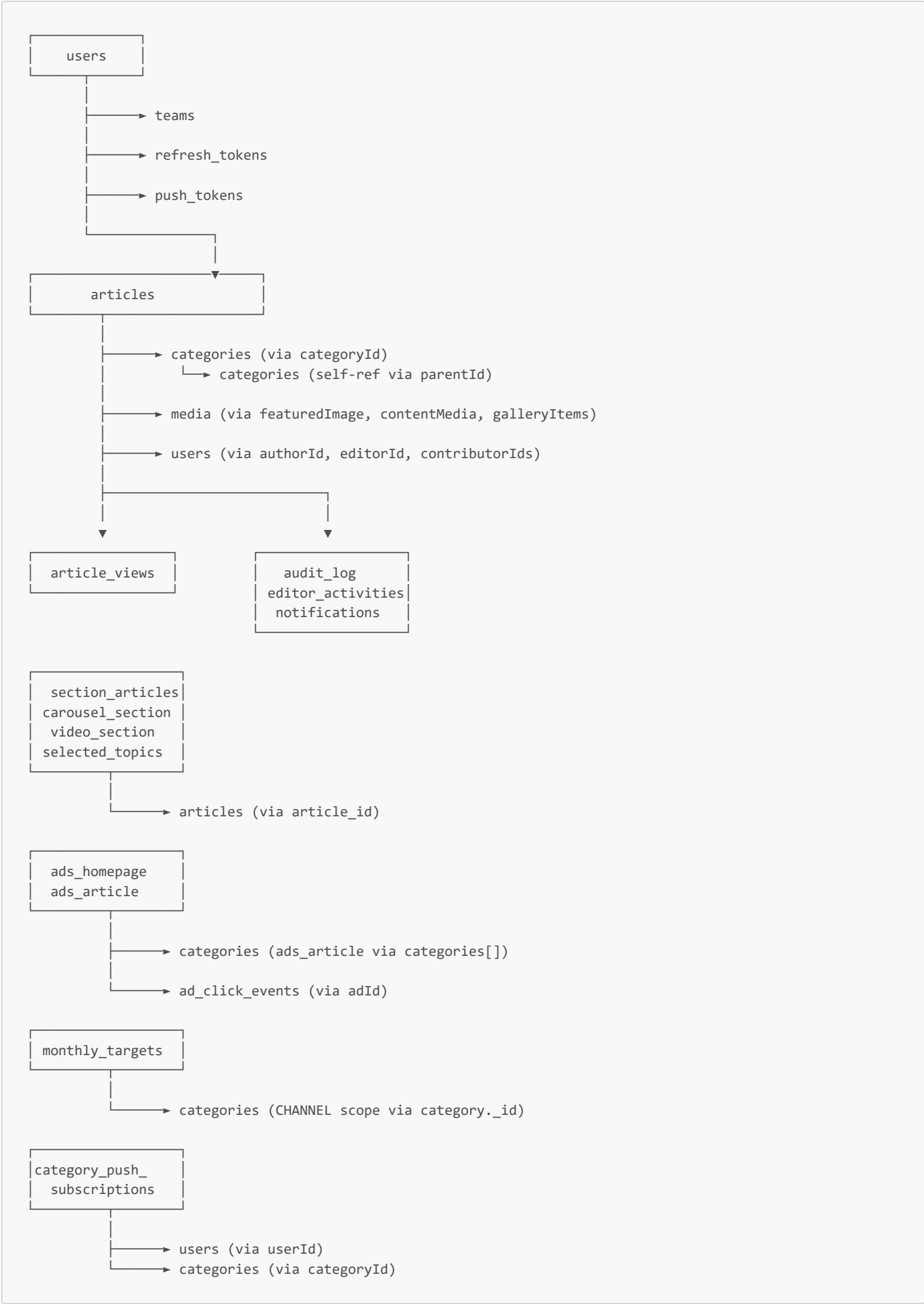
Schema:

```
{
  _id: ObjectId,
  name: string,           // e.g. "Facebook", "Twitter"
  slug: string,           // URL-friendly
  url: string,            // Social media URL
  icon: string            // Icon name
}
```

Indexes:

```
{
  slug: 1;
} // unique
```

3.27 Data Model Diagram



Relasi Utama:

- 1. **Articles** ↔ **Users**: Many-to-Many (author, editor, contributors)
- 2. **Articles** ↔ **Categories**: Many-to-One
- 3. **Articles** ↔ **Media**: One-to-Many
- 4. **Users** ↔ **Teams**: Many-to-One
- 5. **Categories** ↔ **Categories**: Self-reference (nested)
- 6. **Ads** ↔ **Categories**: Many-to-Many (ads_article)

Denormalization Strategy:

- **articles**: Denormalize `category`, `author`, `editor`, `contributors` untuk performa read
- **section_articles**, **carousel_section**: Denormalize `article` snapshot
- **notifications**, **audit_log**, **editor_activities**: Denormalize `actor`, `receiver` info
- Trade-off: Faster reads, slower writes, eventual consistency

Cascade Delete Rules:

- **Soft Delete**: users, articles, ads (set `deletedAt`)
- **Hard Delete**: media (orphaned files cleanup job), refresh_tokens (expired), article_views (retention policy)
- **No Cascade**: audit_log, editor_activities (historical data tetap ada)

4. API Layer

4.1 REST API Structure

/api/	
├─ auth/	# Authentication & session
│ ├─ login/	# POST - Email/password login
│ ├─ logout/	# POST - Logout & clear cookies
│ ├─ refresh/	# POST - Refresh access token
│ ├─ me/	# GET - Current user info
│ └─ register/	# POST - Register new user (admin only)
├─ articles/	# Article CRUD & workflows
│ ├─ route.ts	# GET (list), POST (create)
│ ├─ [id]/	# GET, PUT, DELETE single article
│ ├─ slug/[slug]/	# GET by slug (public)
│ ├─ approval/	# POST - Approve/reject article
│ ├─ publish/	# POST - Publish article
│ ├─ schedule/	# POST - Schedule article
│ ├─ takedown/	# POST - Takedown published article
│ ├─ related/	# GET - Related articles
│ └─ autosave/	# POST - Autosave draft
├─ categories/	# Category management
│ ├─ route.ts	# GET (list), POST (create)
│ ├─ [id]/	# GET, PUT, DELETE single category
│ ├─ slug/[slug]/	# GET by slug (public)
│ ├─ tree/	# GET - Category tree (nested)
│ └─ featured/	# GET - Featured categories
├─ users/	# User management
│ ├─ route.ts	# GET (list), POST (create)
│ ├─ [id]/	# GET, PUT, DELETE single user
│ ├─ slug/[slug]/	# GET by slug (public author profile)
│ └─ change-password/	# POST - Change password
├─ teams/	# Team management
│ ├─ route.ts	# GET (list), POST (create)
│ └─ [id]/	# GET, PUT, DELETE
├─ media/	# Media upload & management
│ ├─ route.ts	# GET (list), POST (create)
│ └─ [id]/	# GET, DELETE single media

└─ presigned-url/	# POST - Get S3 presigned URL
└─ upload/	# POST - Upload media to S3
└─ cleanup/	# DELETE - Cleanup orphaned media
└─ analytics/	# Analytics data
└─ │ audience/	# GET - Audience analytics
└─ │ author/	# GET - Author performance
└─ │ workflow/	# GET - Workflow analytics
└─ │ writing/	# GET - Writing performance
└─ │ view-article/	# POST - Track article view
└─ ads/	# Advertisement management
└─ │ homepage/	# CRUD homepage ads
└─ │ single-article/	# CRUD article ads
└─ │ click/	# POST - Track ad click
└─ │ history/	# GET - Ad performance history
└─ sponsor/	# Sponsor management
└─ │ route.ts	# GET (list), POST (create)
└─ │ [id]/	# GET, PUT, DELETE
└─ configuration/	# Site configuration
└─ │ route.ts	# GET (all), POST/PUT (update)
└─ │ [key]/	# GET by key
└─ monthly-target/	# Monthly KPI targets
└─ │ route.ts	# GET (list), POST (create)
└─ │ [id]/	# GET, PUT, DELETE
└─ notification/	# Notification management
└─ │ route.ts	# GET (list user notifications)
└─ │ [id]/read/	# PUT - Mark as read
└─ │ send/	# POST - Send notification (admin)
└─ push-token/	# FCM token management
└─ │ register/	# POST - Register FCM token
└─ │ unregister/	# DELETE - Unregister token
└─ │ category-subscribe/	# POST - Subscribe to category
└─ publish-scheduled/	# Scheduled publish cron job
└─ │ route.ts	# POST - Publish scheduled articles (cron)
└─ refactoring-articles/	# Data migration & refactoring
└─ │ denormalization/	# POST - Denormalize articles
└─ reports/	# Reporting endpoints
└─ │ kpi-user/	# GET - User KPI report
└─ │ article-writer/	# GET - Writer performance report
└─ search/	# Search endpoints
└─ │ route.ts	# GET - Full-text search
└─ │ suggestions/	# GET - Search suggestions
└─ selected-topics/	# Selected topics (homepage)
└─ │ route.ts	# GET (list), POST (create)
└─ │ [id]/	# GET, PUT, DELETE
└─ sitemap/	# Sitemap generation
└─ │ articles/	# GET - Articles sitemap XML
└─ │ categories/	# GET - Categories sitemap XML
└─ │ authors/	# GET - Authors sitemap XML
└─ social-link/	# Social links
└─ │ route.ts	# GET (list), POST (create)
└─ │ [id]/	# GET, PUT, DELETE
└─ tags/	# Tag management
└─ │ recommendations/	# GET - Tag recommendations
└─ │ popular/	# GET - Popular tags

```
|
└─ indeks/                # News index
   └─ route.ts            # GET - All articles index
```

4.2 API Design Patterns

Request/Response Structure

Success Response:

```
{
  "data": {
    /* payload */
  },
  "meta": {
    "page": 1,
    "limit": 10,
    "total": 100,
    "nextCursor": "cursor_string"
  }
}
```

Error Response:

```
{
  "error": "User-friendly error message",
  "code": "ERROR_CODE",
  "details": {
    /* optional debug info */
  }
}
```

HTTP Status Codes:

- 200 OK - Success
- 201 Created - Resource created
- 400 Bad Request - Validation error
- 401 Unauthorized - Auth required
- 403 Forbidden - Permission denied
- 404 Not Found - Resource not found
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Pagination Strategy

Cursor-based Pagination (untuk listing artikel, user, dll):

```
// Request
GET /api/articles?limit=10&cursor=abc123

// Response
{
  "data": [...],
  "meta": {
    "nextCursor": "xyz789" // null jika sudah habis
  }
}

// Implementation
const articles = await collection
  .find(query)
```

```
.sort({ publishedAt: -1, _id: -1 })
.limit(limit + 1) // fetch N+1
.toArray();

const hasMore = articles.length > limit;
const items = hasMore ? articles.slice(0, limit) : articles;
const nextCursor = hasMore ? encodeCursor(items[items.length - 1]) : null;
```

Keuntungan cursor-based:

- Konsisten meski data berubah (no offset drift)
- Scalable untuk dataset besar
- Performa stabil (no full scan)

Offset-based Pagination (untuk admin panel):

```
// Request
GET /api/articles?page=2&limit=10

// Response
{
  "data": [...],
  "meta": {
    "page": 2,
    "limit": 10,
    "total": 100,
    "totalPages": 10
  }
}
```

Filtering & Sorting

Query Parameters:

```
GET /api/articles?
  status=PUBLISHED&
  categorySlug=nasional&
  authorId=123&
  search=keyword&
  sort=-publishedAt& // "-" untuk descending
  limit=10&
  cursor=abc123
```

Implementation:

```
const query: any = {};
if (status) query.status = status;
if (categorySlug) query["category.slug"] = categorySlug;
if (authorId) query.authorId = new ObjectId(authorId);
if (search) {
  query.$text = { $search: search };
}

const sort: any = {};
if (sortParam === "-publishedAt") {
  sort.publishedAt = -1;
  sort._id = -1; // tie-breaker
}
```

Rate Limiting

Strategy: Token bucket per IP + per user

- Anonymous: 100 req/min
- Authenticated: 300 req/min
- Admin: 1000 req/min

Implementation (conceptual, not yet implemented):

```
import rateLimit from "express-rate-limit";

const limiter = rateLimit({
  windowMs: 60 * 1000, // 1 minute
  max: 100, // 100 requests
  standardHeaders: true,
  legacyHeaders: false,
});
```

4.3 Authentication & Authorization Flow

Flow Diagram



JWT Token Structure

Access Token (short-lived: 15 min):

```
{
  "_id": "user_id",
  "name": "John Doe",
```

```
{
  "email": "john@example.com",
  "role": "editor",
  "slug": "john-doe",
  "avatar": "https://...",
  "teamId": "team_id",
  "iat": 1234567890,
  "exp": 1234568790
}
```

Refresh Token (long-lived: 30 days):

- Stored hashed di `refresh_tokens` collection
- Rotation strategy: setiap refresh menghasilkan token baru
- Old token di-revoke (set `revokedAt`)

Middleware Chain

```
// src/app/api/articles/route.ts
export async function GET(request: NextRequest) {
  // 1. Extract & verify token
  const user = await getUserFromRequest(request);
  if (!user) {
    return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
  }

  // 2. Check permission
  if (!hasPermission(user.role, "view_content")) {
    return NextResponse.json({ error: "Forbidden" }, { status: 403 });
  }

  // 3. Business logic
  const articles = await getArticles({ userId: user._id });
  return NextResponse.json({ data: articles });
}
```

Permission Checking

```
// src/lib/auth-client.ts
export const ROLE_PERMISSIONS: Record<string, Permission[]> = {
  admin: ["all"],
  "editor-in-chief": [
    "create_article",
    "edit_any_article",
    "publish_article",
    "manage_users",
    "view_analytics",
    // ... 20+ permissions
  ],
  editor: [
    "create_article",
    "edit_any_article",
    "approve_article",
    "reject_article",
    // ...
  ],
  writer: [
    "create_article",
    "edit_own_article",
    "submit_article",
    "view_own_analytics",
  ],
  // ...
};
```



```
export function hasPermission(role: string, permission: Permission): boolean {
  const permissions = ROLE_PERMISSIONS[role] || [];
  return permissions.includes("all") || permissions.includes(permission);
}
```

4.4 API Endpoints Reference

Dokumentasi API lengkap tersedia dalam format OpenAPI Swagger 3.0.

 Lihat file: `api.yaml`

File `api.yaml` berisi spesifikasi lengkap untuk:

- **8 kategori endpoint:** Auth, Articles, Categories, Users, Media, Analytics, Search, Notifications
- **Request/response schemas:** Semua data model (Article, User, Category, Media, Notification, dll)
- **Authentication:** Bearer token & cookie-based auth
- **Query parameters:** Filter, pagination, search
- **Error responses:** Standard error format

Preview API Documentation:

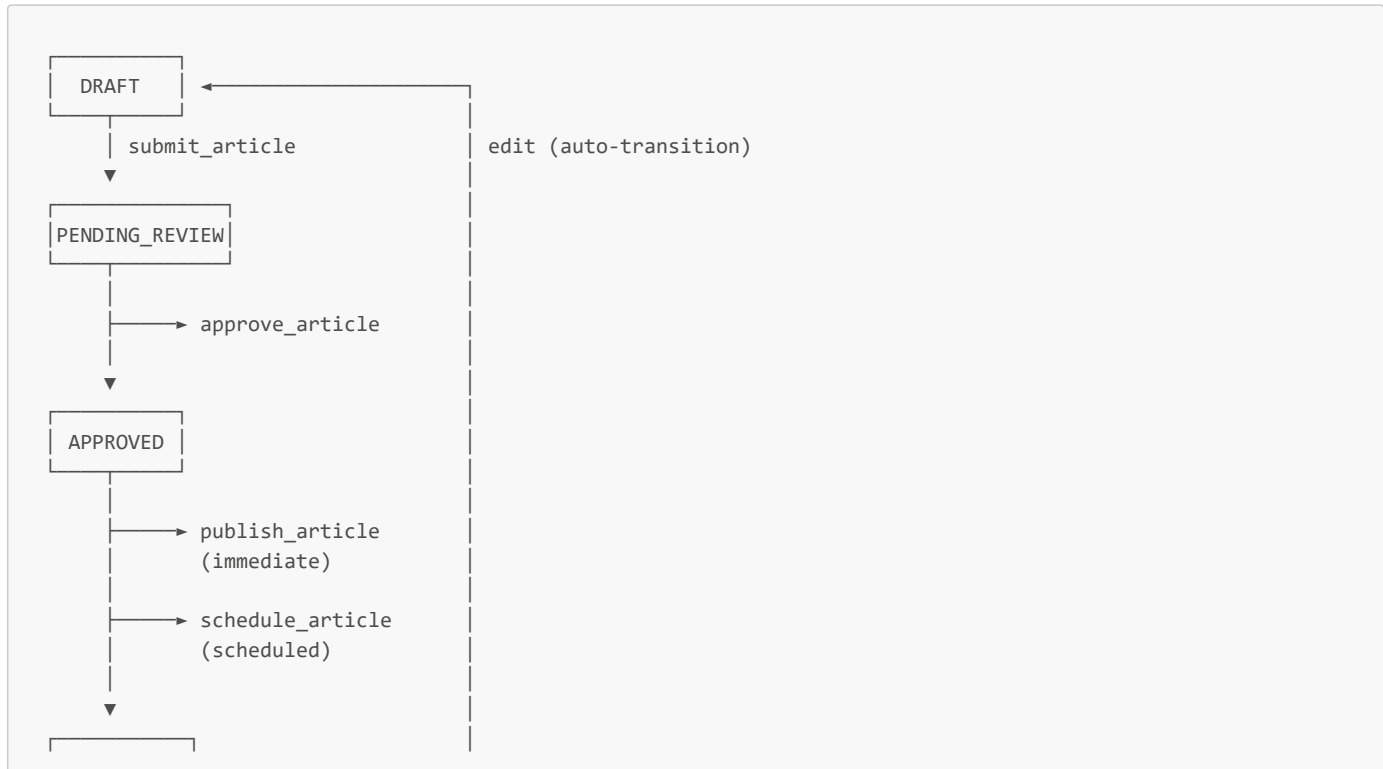
- Buka `api.yaml` dengan [Swagger Editor](#)
- Import ke Postman/Insomnia untuk testing
- Generate client SDK dengan OpenAPI Generator

Key Endpoints:

- Auth: `/api/auth/*` (login, logout, refresh, me)
- Articles: `/api/articles/*` (CRUD, approval, publish, schedule, takedown)
- Categories: `/api/categories/*` (CRUD, tree, featured)
- Users: `/api/users/*` (CRUD, author profile)
- Media: `/api/media/*` (upload, presigned URL, cleanup)
- Analytics: `/api/analytics/*` (audience, author, workflow, view tracking)
- Search: `/api/search/*` (full-text, suggestions)
- Notifications: `/api/notification/*` (list, mark read, send)

5. Business Logic & Domain Models

5.1 Article Workflow State Machine





Status Enum:

```
export enum ArticleStatus {
  DRAFT = "DRAFT", // Artikel sedang ditulis
  PENDING_REVIEW = "PENDING_REVIEW", // Submitted untuk review
  PUBLISHED = "PUBLISHED", // Sudah terbit
  SCHEDULED = "SCHEDULED", // Dijadwalkan terbit
  REJECTED = "REJECTED", // Ditolak editor
  TAKEN_DOWN = "TAKEN_DOWN", // Artikel ditarik dari publikasi
  DELETED = "DELETED", // Soft deleted
}
```

Status Transitions (dari src/types/article.ts):

```
export const STATUS_ROLE_MAP: Record<ArticleStatus, string[]> = {
  DRAFT: [
    "reporter",
    "writer",
    "contributor",
    "editor",
    "head-of",
    "managing-editor",
    "editor-in-chief",
    "admin",
  ],
  PENDING_REVIEW: [
    "reporter",
    "writer",
    "contributor",
    "editor",
    "head-of",
  ],
}
```

```

    "managing-editor",
    "editor-in-chief",
    "admin",
  ],
  REJECTED: [
    "editor",
    "head-of",
    "managing-editor",
    "editor-in-chief",
    "admin",
  ],
  PUBLISHED: [
    "editor",
    "head-of",
    "managing-editor",
    "editor-in-chief",
    "admin",
  ],
  SCHEDULED: [
    "editor",
    "head-of",
    "managing-editor",
    "editor-in-chief",
    "admin",
  ],
  TAKEN_DOWN: [
    "editor",
    "head-of",
    "managing-editor",
    "editor-in-chief",
    "admin",
  ],
  DELETED: ["editor-in-chief", "admin"],
};

```

Automatic Transitions:

1. **Edit after submit:** Jika artikel dengan status `PENDING_REVIEW`, `APPROVED`, atau `REJECTED` di-edit, status otomatis kembali ke `PENDING_REVIEW`
2. **Scheduled publish:** Cron job (`/api/publish-scheduled`) mengubah `SCHEDULED` → `PUBLISHED` saat `scheduledAt` terpenuhi

Notification Triggers (dari `src/types/article.ts`):

```

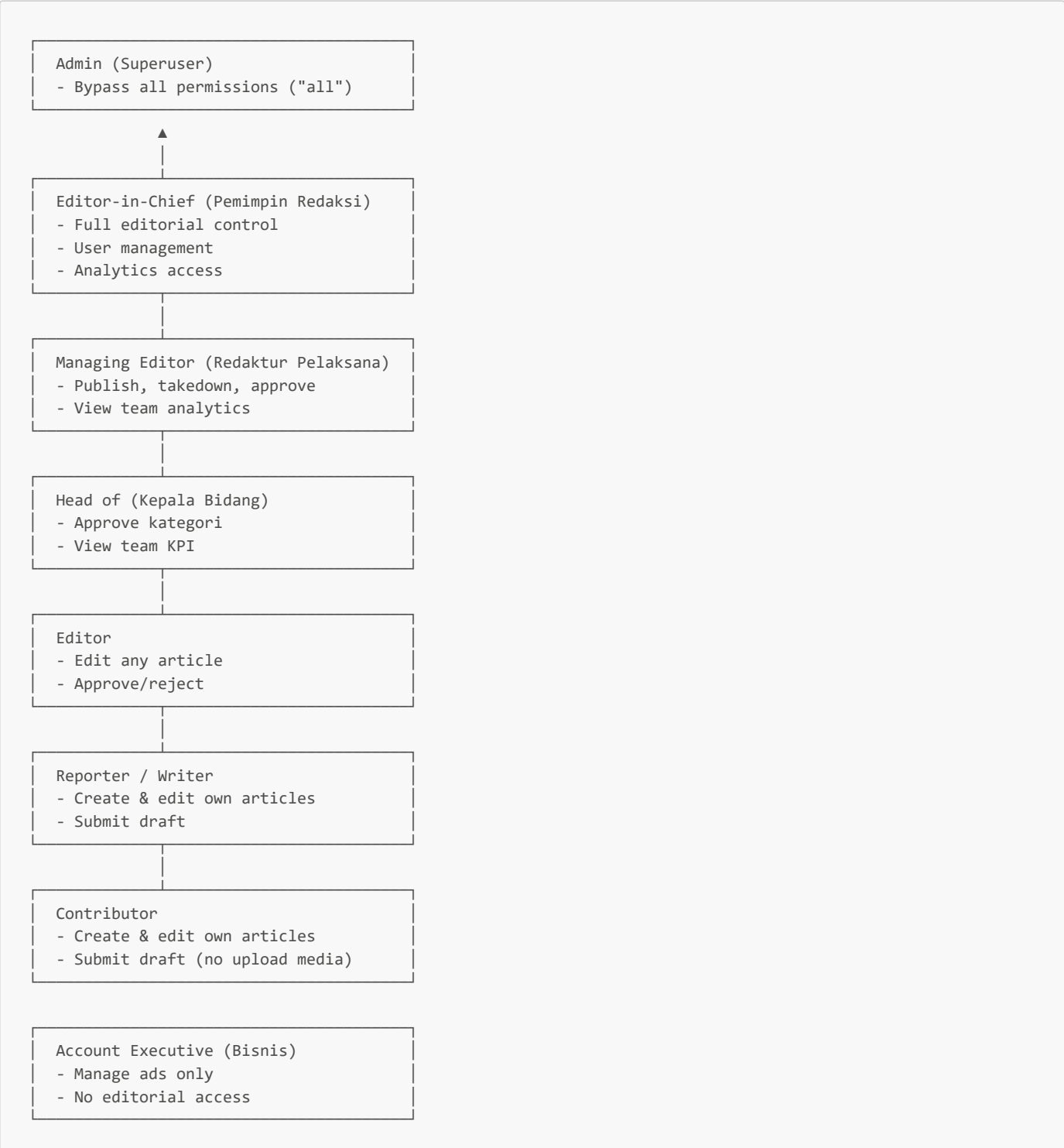
export const STATUS_NOTIFICATION: Record<
  ArticleStatus,
  {
    roles: string[];
    getMessage: (article: Article, reason?: string) => string;
  }
> = {
  PENDING_REVIEW: {
    roles: ["editor", "head-of"],
    getMessage: (a) => `Artikel "${a.title}" menunggu review.`,
  },
  REJECTED: {
    roles: ["reporter", "writer", "contributor"],
    getMessage: (a, r) =>
      `Artikel "${a.title}" ditolak.${r ? " Alasan: " + r : ""}`,
  },
  PUBLISHED: {
    roles: ["reporter", "writer", "contributor"],
    getMessage: (a) => `Artikel "${a.title}" telah dipublikasikan.`,
  },
  SCHEDULED: {
    roles: ["reporter", "writer", "contributor"],
    getMessage: (a) => `Artikel "${a.title}" dijadwalkan terbit.`,
  },
};

```

```
TAKEN_DOWN: {
  roles: ["reporter", "writer", "contributor"],
  getMessage: (a, r) =>
    `Artikel "${a.title}" di-takedown.${r ? " Alasan: " + r : ""}`,
},
DELETED: {
  roles: ["reporter", "writer", "contributor"],
  getMessage: (a, r) =>
    `Artikel "${a.title}" dihapus.${r ? " Alasan: " + r : ""}`,
},
DRAFT: { roles: [], getMessage: () => "" }, // No notification
};
```

5.2 Role-Based Access Control (RBAC)

Roles Hierarchy



Subscriber (Pelanggan)
- View content, comment

Permission Matrix

Permissions (dari `src/lib/auth-client.ts`):

- **all** - Superuser bypass (admin only)
- **Article CRUD:** `create_article`, `edit_own_article`, `edit_any_article`, `delete_own_article`, `delete_any_article`
- **Article Workflow:** `submit_article`, `approve_article`, `reject_article`, `publish_article`, `schedule_article`, `takedown_article`, `restore_article`
- **Taxonomy:** `manage_categories`, `manage_tags`
- **Media:** `upload_media`, `delete_own_media`, `delete_any_media`
- **Users:** `view_users`, `manage_users`, `manage_roles`
- **Editorial:** `manage_editorial`
- **Analytics:** `view_analytics`, `view_team_analytics`, `view_own_analytics`
- **Ads:** `manage_ads`, `view_ad_analytics`
- **System:** `view_audit_logs`, `send_push_notifications`, `manage_settings`
- **General:** `view_content`, `comment`

Role → Permissions Mapping:

Permission	Admin	Editor-in-Chief	Managing Editor	Head of	Editor	Writer	Reporter	Contributor	Account Exec	Subscriber
all (bypass)	☑	✗	✗	✗	✗	✗	✗	✗	✗	✗
create_article	☑	☑	☑	☑	☑	☑	☑	☑	✗	✗
edit_own_article	☑	☑	☑	☑	☑	☑	☑	☑	✗	✗
edit_any_article	☑	☑	☑	☑	☑	✗	✗	✗	✗	✗
delete_own_article	☑	☑	☑	☑	☑	☑	☑	✗	✗	✗
delete_any_article	☑	☑	☑	✗	✗	✗	✗	✗	✗	✗
submit_article	☑	☑	☑	☑	☑	☑	☑	☑	✗	✗
approve_article	☑	☑	☑	☑	☑	✗	✗	✗	✗	✗
reject_article	☑	☑	☑	☑	☑	✗	✗	✗	✗	✗
publish_article	☑	☑	☑	✗	✗	✗	✗	✗	✗	✗
schedule_article	☑	☑	☑	☑	✗	✗	✗	✗	✗	✗
takedown_article	☑	☑	☑	✗	✗	✗	✗	✗	✗	✗
manage_categories	☑	☑	☑	✗	✗	✗	✗	✗	✗	✗
manage_tags	☑	☑	☑	☑	✗	✗	✗	✗	✗	✗
upload_media	☑	☑	☑	☑	☑	☑	☑	☑	✗	✗
delete_any_media	☑	☑	☑	✗	✗	✗	✗	✗	✗	✗
manage_users	☑	☑	✗	✗	✗	✗	✗	✗	✗	✗
view_analytics	☑	☑	☑	✗	✗	✗	✗	✗	✗	✗
view_team_analytics	☑	☑	☑	☑	✗	✗	✗	✗	✗	✗
view_own_analytics	☑	☑	☑	☑	☑	☑	☑	✗	✗	✗
manage_ads	☑	✗	✗	✗	✗	✗	✗	✗	☑	✗

Permission	Admin	Editor-in-Chief	Managing Editor	Head of	Editor	Writer	Reporter	Contributor	Account Exec	Subscriber
view_ad_analytics	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒
send_push_notifications	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒
view_audit_logs	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒
view_content	☒	☒	☒	☒	☒	☒	☒	☒	☒	☒

Permission Checking Logic:

```
// src/lib/auth-client.ts
export function hasPermission(
  userRole: string,
  permission: Permission,
): boolean {
  const permissions = ROLE_PERMISSIONS[userRole] || [];
  return permissions.includes("all") || permissions.includes(permission);
}

// Usage in API route
export async function PUT(
  request: NextRequest,
  { params }: { params: { id: string } },
) {
  const user = await getUserFromRequest(request);
  if (!user) {
    return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
  }

  // Check ownership
  const article = await getArticleById(params.id);
  const isOwner = article.authorId === user._id;

  // Check permission
  const canEditOwn = hasPermission(user.role, "edit_own_article");
  const canEditAny = hasPermission(user.role, "edit_any_article");

  if (!canEditAny && (!canEditOwn || !isOwner)) {
    return NextResponse.json({ error: "Forbidden" }, { status: 403 });
  }

  // Proceed with update
  // ...
}
```

5.3 Status Transition Rules

Who Can Change What Status:

From Status	To Status	Required Permission	Notes
DRAFT	PENDING_REVIEW	submit_article	Penulis submit untuk review
PENDING_REVIEW	APPROVED	approve_article	Editor approve
PENDING_REVIEW	REJECTED	reject_article	Editor reject
APPROVED	PUBLISHED	publish_article	Publish immediately
APPROVED	SCHEDULED	schedule_article	Schedule publish
SCHEDULED	PUBLISHED	Cron job	Automatic via /api/publish-scheduled

From Status	To Status	Required Permission	Notes
PUBLISHED	TAKEN_DOWN	takedown_article	Takedown published article
TAKEN_DOWN	DRAFT	restore_article	Restore untuk re-edit
REJECTED	DRAFT	Auto on edit	Penulis edit setelah reject
*	DELETED	Admin/EIC only	Hard delete (soft delete via <code>deletedAt</code>)

Automatic Status Transitions:

- 1. **Edit after review:** Artikel dengan status `PENDING_REVIEW`, `APPROVED`, atau `REJECTED` yang di-edit otomatis kembali ke `PENDING_REVIEW`
- 2. **Scheduled publish:** Cron job setiap menit cek `articles` dengan `status=SCHEDULED` dan `scheduledAt <= now()`, lalu ubah ke `PUBLISHED`

Status Validation Logic:

```
// src/services/article/writeArticleService.ts
async function validateStatusTransition(
  currentStatus: ArticleStatus,
  newStatus: ArticleStatus,
  userRole: string,
): Promise<{ valid: boolean; error?: string }> {
  // Check if role allowed to set this status
  const allowedRoles = STATUS_ROLE_MAP[newStatus];
  if (!allowedRoles.includes(userRole) && !hasPermission(userRole, "all")) {
    return { valid: false, error: "Insufficient permission for this status" };
  }

  // Validate transition logic
  const validTransitions: Record<ArticleStatus, ArticleStatus[]> = {
    DRAFT: ["PENDING_REVIEW", "PUBLISHED", "SCHEDULED"], // Writer+ can submit or admin can publish directly
    PENDING_REVIEW: ["APPROVED", "REJECTED", "PUBLISHED", "SCHEDULED"], // Editor+ can
    approve/reject/publish
    APPROVED: ["PUBLISHED", "SCHEDULED"], // Editor+ can publish or schedule
    SCHEDULED: ["PUBLISHED"], // Cron job only
    PUBLISHED: ["TAKEN_DOWN"], // Editor+ can takedown
    TAKEN_DOWN: ["DRAFT", "PUBLISHED"], // Restore to draft or re-publish
    REJECTED: ["DRAFT", "PENDING_REVIEW"], // Re-edit or re-submit
    DELETED: [], // Terminal state
  };

  if (!validTransitions[currentStatus]?.includes(newStatus)) {
    return {
      valid: false,
      error: `Invalid transition from ${currentStatus} to ${newStatus}`,
    };
  }

  return { valid: true };
}
```

5.4 Audit Logging

What Gets Logged (dari `src/services/auditLogService.ts`):

- Article CRUD operations (create, update, delete, publish, takedown, etc)
- Status changes (with `statusFrom` and `statusTo`)
- Section management (carousel, featured, headline, popular, editor choice)
- User management (create, update, delete, role change)
- Category management
- Configuration changes
- Ads management

Log Entry Structure:

```
{
  _id: ObjectId,
  actor: {
    _id: ObjectId,
    name: string,
    email: string,
    avatarUrl?: string
  },
  action: "CREATE" | "UPDATE" | "DELETE" | "PUBLISH" | "SCHEDULE" | "TAKE_DOWN" | "REJECT" | "RESTORE",
  entity: "articles" | "users" | "categories" | "section_featured" | ...,
  entityId: ObjectId,
  details?: string,
  oldValue?: any,
  newValue?: any,
  meta?: {
    statusFrom?: ArticleStatus,
    statusTo?: ArticleStatus,
    articleTitle?: string,
    reason?: string,
    // ...
  },
  createdAt: Date,
  ipAddress?: string
}
```

Logging Service Usage:

```
import { logAudit } from "@services/auditLogService";

// Example: Log article publish
await logAudit({
  actor: {
    _id: user._id,
    name: user.name,
    email: user.email,
    avatarUrl: user.avatar,
  },
  action: "PUBLISH",
  entity: "articles",
  entityId: article._id,
  details: `Published article "${article.title}"`,
  meta: {
    statusFrom: "APPROVED",
    statusTo: "PUBLISHED",
    articleTitle: article.title,
  },
  ipAddress: request.ip,
});
```

Log Retention Policy:

- **Retention:** Unlimited (historical data)
- **Indexing:** By `entityId`, `actor._id`, `createdAt`
- **Query Performance:** Compound indexes untuk common queries
- **Export:** Admin dapat export audit trail sebagai CSV

Audit Trail Queries:

```
// Get all actions on an article
const logs = await auditLogCollection
  .find({ entity: "articles", entityId: new ObjectId(articleId) })
```



```

.sort({ createdAt: -1 })
.toArray();

// Get all actions by a user
const logs = await auditLogCollection
  .find({ "actor._id": new ObjectId(userId) })
  .sort({ createdAt: -1 })
  .limit(100)
  .toArray();

// Get status changes for an article
const statusChanges = await auditLogCollection
  .find({
    entity: "articles",
    entityId: new ObjectId(articleId),
    "meta.statusFrom": { $exists: true },
  })
  .sort({ createdAt: 1 })
  .toArray();

```

5.5 Notification System

Notification Types (dari `src/types/notification.ts`):

- `article-submitted` - Artikel disubmit untuk review
- `article-published` - Artikel dipublikasikan
- `article-revision` - Artikel perlu revisi
- `article-rejected` - Artikel ditolak
- `article-taken-down` - Artikel di-takedown
- `article-deleted` - Artikel dihapus
- `article-raising` - Artikel di-raise (promoted)
- `article-approval` - Artikel di-approve
- `schedule-published` - Scheduled article dipublish
- `ads-raised` - Ads di-raise
- `ads-taken-down` - Ads di-takedown
- `system-announcement` - System announcement dari admin

Delivery Channels:

1. **In-App:** MongoDB `notifications` collection, polling via API
2. **Push (FCM):** Firebase Cloud Messaging untuk web/mobile push
3. **Email:** (Not yet implemented)

Notification Routing Logic:

```

// src/services/notificationService.ts
export async function sendNotification({
  receiver,
  actor,
  type,
  title,
  message,
  targetId,
  link,
  sendPush = false,
}): {
  receiver: NotificationActor;
  actor: NotificationActor;
  type: NotificationType;
  title: string;
  message: string;
  targetId?: string;
  link?: string;
  sendPush?: boolean;
}

```

```

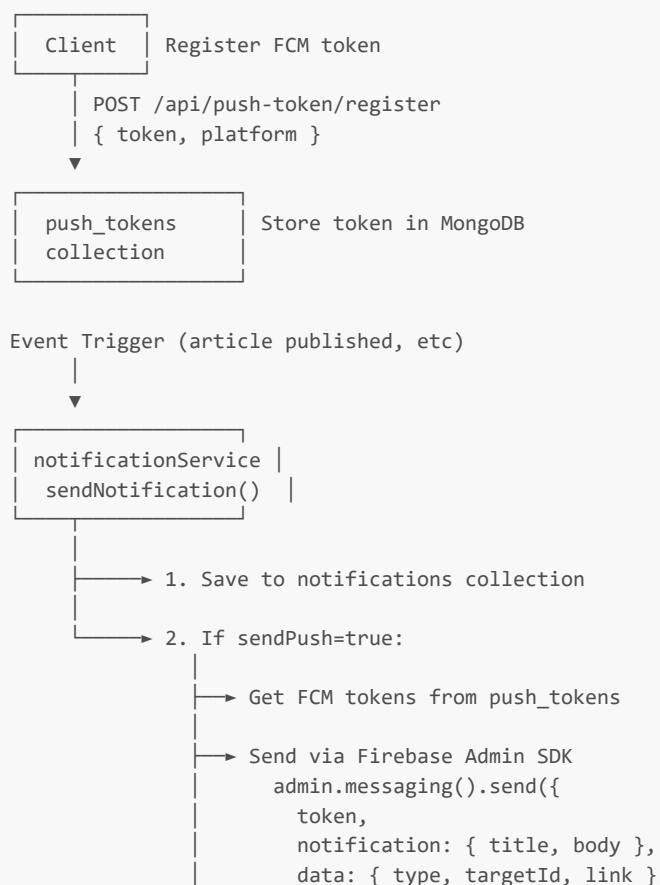
    }) {
      // 1. Save to database
      const notification = await notificationsCollection.insertOne({
        receiver,
        actor,
        type,
        title,
        message,
        targetId,
        link,
        isPushSent: false,
        readAt: null,
        createdAt: new Date(),
      });

      // 2. Send push notification if enabled
      if (sendPush) {
        await sendPushNotification({
          userId: receiver._id,
          title,
          body: message,
          data: {
            type,
            targetId,
            link,
          },
        });

        await notificationsCollection.updateOne(
          { _id: notification.insertedId },
          { $set: { isPushSent: true } },
        );
      }
    }
  }
}

```

Push Notification Flow:



```
    })  
    → Mark isPushSent=true
```

User Preferences (Not yet implemented):

- User dapat opt-out dari notification types tertentu
- Category-based subscription via `category_push_subscriptions`
- Frequency control (immediate, digest daily, digest weekly)

Notification Cleanup:

- Read notifications older than 30 days di-archive
- Unread notifications tetap ada (no auto-delete)

6. Infrastruktur & External Services

6.1 MongoDB Atlas

Cluster Configuration:

- **Tier:** M10+ (production), M0/M2 (development)
- **Region:** Sesuai deployment (Asia Pacific untuk latency rendah)
- **Replica Set:** 3-node replica set untuk high availability
- **Version:** MongoDB 7.1+

Connection String:

```
mongodb+srv://<username>:<password>@<cluster>.mongodb.net/<dbname>?retryWrites=true&w=majority
```

Connection Pooling (dari `src/lib/db/db.ts`):

```
import { MongoClient, Db } from "mongodb";  
  
let client: MongoClient | null;  
let db: Db | null;  
  
export async function connectToDatabase(): Promise<Db> {  
  if (db) return db; // Reuse existing connection  
  
  if (!process.env.MONGO_URL) {  
    throw new Error("MONGO_URL environment variable not set");  
  }  
  
  if (!client) {  
    client = new MongoClient(process.env.MONGO_URL);  
    await client.connect();  
  }  
  
  db = client.db(process.env.DB_NAME || "arasvara_news");  
  return db;  
}
```

Connection Pool Settings:

- **Max Pool Size:** 100 (default)
- **Min Pool Size:** 10
- **Connection Timeout:** 30s
- **Socket Timeout:** 360s

Indexes & Performance:

- **Text Indexes:** `articles` collection untuk full-text search

- **Compound Indexes:** { status: 1, publishedAt: -1 } untuk listing
- **Sparse Indexes:** { publicPath: 1 } untuk structured URL lookup
- **TTL Indexes:** { expiresAt: 1 } pada refresh_tokens untuk auto-cleanup
- **Index Coverage:** Query plans di-monitor via MongoDB Atlas Performance Advisor

Performance Tuning:

```
// Projection untuk mengurangi data transfer
db.articles
  .find(
    { status: "PUBLISHED" },
    { projection: { content: 0, revisionHistory: 0 } }, // Exclude heavy fields
  )
  .limit(10);

// Aggregation pipeline untuk denormalization
db.articles.aggregate([
  { $match: { status: "PUBLISHED" } },
  {
    $lookup: {
      from: "users",
      localField: "authorId",
      foreignField: "_id",
      as: "author",
    },
  },
  { $unwind: "$author" },
  { $project: { "author.password": 0 } }, // Exclude sensitive fields
]);
```

Backup Strategy:

- **Automated Snapshots:** Daily snapshots via MongoDB Atlas (retention: 7 days)
- **Point-in-Time Recovery:** Enabled untuk cluster M10+
- **Export:** Weekly export ke S3 untuk disaster recovery
- **Restore Testing:** Monthly restore test di staging environment

Monitoring:

- **Metrics:** CPU, memory, disk I/O, network via MongoDB Atlas dashboard
- **Slow Queries:** Profiler enabled untuk queries > 100ms
- **Alerts:** Email/Slack alert untuk connection spike, disk usage > 80%

6.2 AWS S3

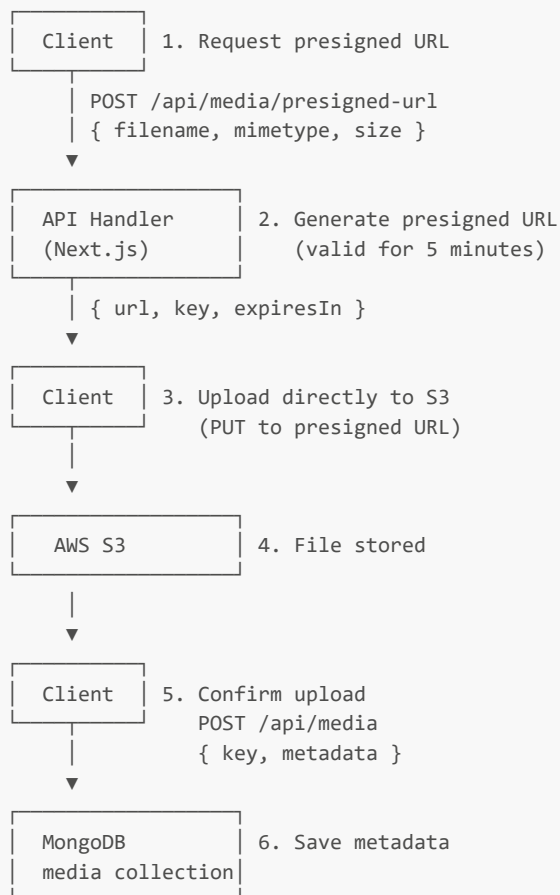
Bucket Structure:

```
s3://arasvara-media/
├── articles/
│   ├── {article-id}/
│   │   ├── images/
│   │   │   ├── {ulid}.webp
│   │   │   └── {ulid}-thumb.webp
│   │   └── videos/
│   │       └── {ulid}.mp4
│   └── ...
├── avatars/
│   └── {user-id}.webp
├── configuration/
│   └── {config-key}.{ext}
├── temp/
│   └── {ulid}.{ext} (cleaned up after 24h)
```

S3 Client Configuration (dari `src/lib/db/s3.ts`):

```
import { S3Client } from "@aws-sdk/client-s3";
import { NodeHttpHandler } from "@smithy/node-http-handler";

const s3Client = new S3Client({
  region: process.env.S3_REGION,
  endpoint: process.env.S3_ENDPOINT, // Optional untuk MinIO
  credentials: {
    accessKeyId: process.env.S3_ACCESS_KEY!,
    secretAccessKey: process.env.S3_SECRET_KEY!,
  },
  forcePathStyle: process.env.S3_FORCE_PATH_STYLE === "true", // For MinIO
  requestHandler: new NodeHttpHandler({
    connectionTimeout: 3000,
    socketTimeout: 30000,
    maxSockets: parseInt(process.env.S3_MAX_SOCKETS || "50"),
  }),
});
```

Upload Flow (Presigned URLs):**Presigned URL Generation:**

```
import { PutObjectCommand } from "@aws-sdk/client-s3";
import { getSignedUrl } from "@aws-sdk/s3-request-presigner";

export async function generatePresignedUploadUrl({
  key,
  contentType,
  expiresIn = 300, // 5 minutes
}) {
```

```

}): {
  key: string;
  contentType: string;
  expiresIn?: number;
}) {
  const command = new PutObjectCommand({
    Bucket: process.env.S3_BUCKET_NAME,
    Key: key,
    ContentType: contentType,
    CacheControl: "public, max-age=31536000, immutable", // 1 year cache
  });

  const url = await getSignedUrl(s3Client, command, { expiresIn });
  return { url, key, expiresIn };
}

```

Access Control (IAM Policy):

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:PutObject", "s3:GetObject", "s3:DeleteObject"],
      "Resource": "arn:aws:s3:::arasvara-media/*"
    },
    {
      "Effect": "Allow",
      "Action": "s3:ListBucket",
      "Resource": "arn:aws:s3:::arasvara-media"
    }
  ]
}

```

CDN Integration:

- **CloudFront** (atau equivalent) di depan S3
- **Cache Headers:** `Cache-Control: public, max-age=31536000, immutable`
- **Custom Domain:** `https://cdn.arasvara.com/`
- **HTTPS Only:** Enforce SSL/TLS
- **Geo Restrictions:** Optional (untuk compliance)

Storage Optimization:

- **WebP Conversion:** Semua images di-convert ke WebP (Sharp)
- **Compression:** Lossy compression quality 85% (balance quality vs size)
- **Thumbnail Generation:** Multiple sizes (thumb, medium, large)
- **Lifecycle Policy:** Delete files di `temp/` setelah 1 hari

6.3 Firebase Authentication

Auth Providers:

- **Email/Password:** Primary provider
- **Google OAuth:** (Extensible, not yet implemented)
- **Facebook OAuth:** (Extensible, not yet implemented)

Firebase Client Configuration (dari `src/lib/firebase-client-config.ts`):

```

import { initializeApp, getApps } from "firebase/app";
import { getAuth } from "firebase/auth";

const firebaseConfig = {

```

```
apiKey: process.env.NEXT_PUBLIC_FIREBASE_API_KEY,  
authDomain: process.env.NEXT_PUBLIC_FIREBASE_AUTH_DOMAIN,  
projectId: process.env.NEXT_PUBLIC_FIREBASE_PROJECT_ID,  
storageBucket: process.env.NEXT_PUBLIC_FIREBASE_STORAGE_BUCKET,  
messagingSenderId: process.env.NEXT_PUBLIC_FIREBASE_MESSAGING_SENDER_ID,  
appId: process.env.NEXT_PUBLIC_FIREBASE_APP_ID,  
};  
  
const app =  
  getApps().length === 0 ? initializeApp(firebaseConfig) : getApps()[0];  
export const auth = getAuth(app);
```

Firebase Admin SDK (dari `src/lib/firebaseAdmin.ts`):

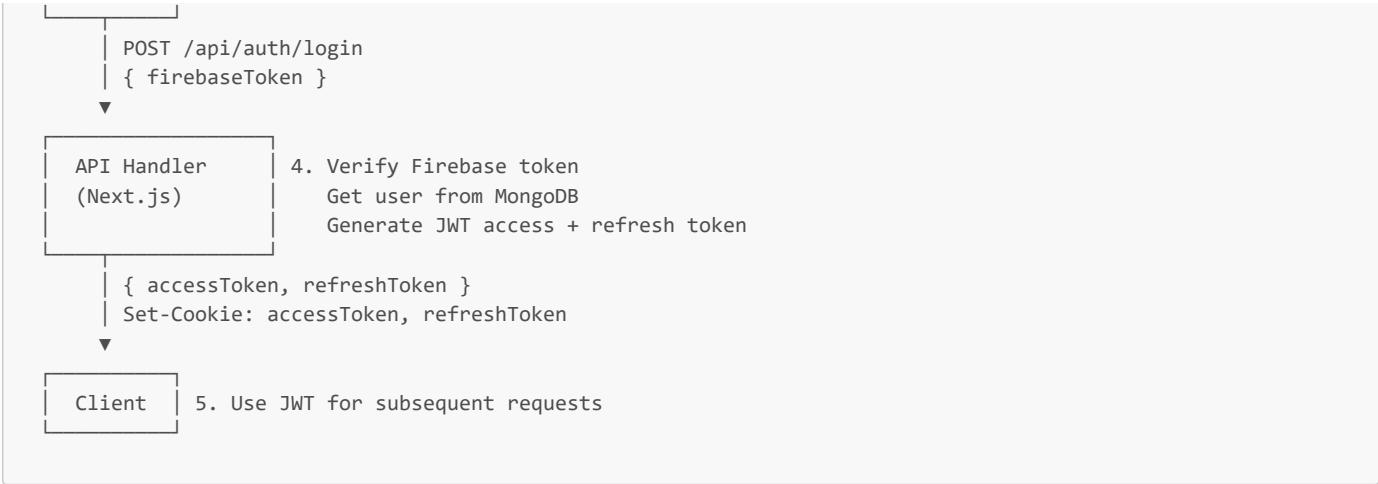
```
import admin from "firebase-admin";  
  
if (!admin.apps.length) {  
  const serviceAccount = JSON.parse(  
    process.env.FIREBASE_SERVICE_ACCOUNT || "{}",  
  );  
  
  admin.initializeApp({  
    credential: admin.credential.cert(serviceAccount),  
  });  
}  
  
export const adminAuth = admin.auth();  
export const adminMessaging = admin.messaging();
```

Custom Claims (Role Injection):

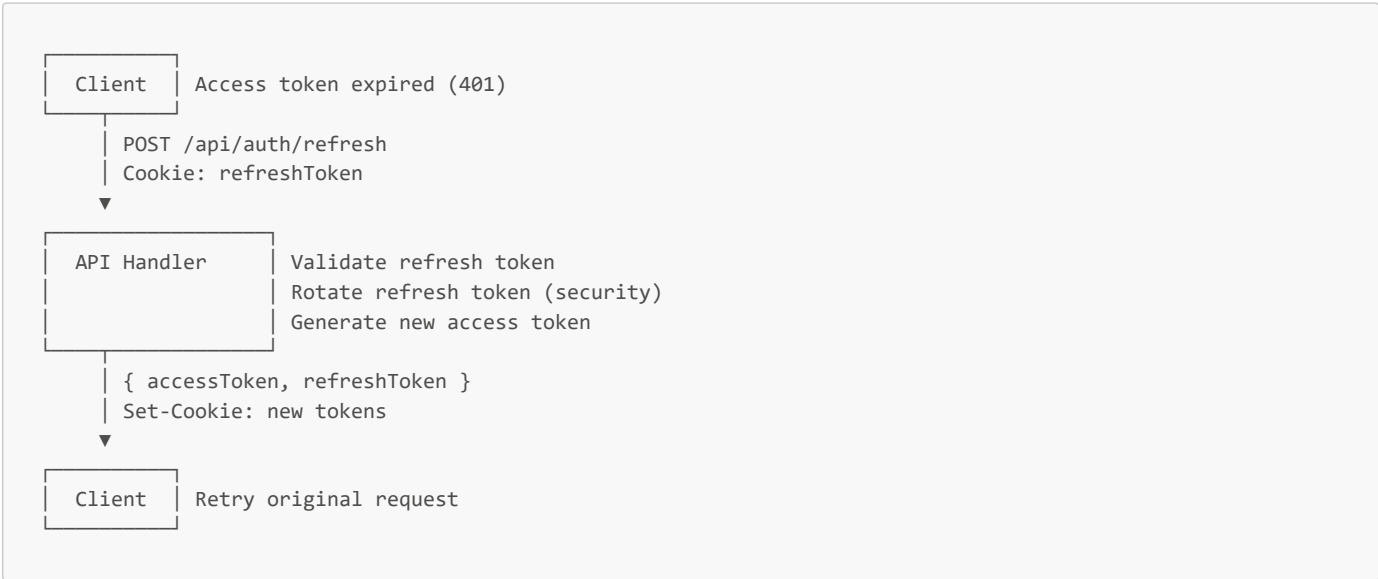
```
// Set custom claims saat user login pertama kali  
import { adminAuth } from "@lib/firebaseAdmin";  
  
export async function setUserClaims(uid: string, claims: { role: string }) {  
  await adminAuth.setCustomUserClaims(uid, claims);  
}  
  
// Verify token dan extract claims  
export async function verifyFirebaseToken(idToken: string) {  
  const decodedToken = await adminAuth.verifyIdToken(idToken);  
  return {  
    uid: decodedToken.uid,  
    email: decodedToken.email,  
    role: decodedToken.role, // Custom claim  
  };  
}
```

Token Flow:





Token Refresh Flow:



Session Management:

- **Access Token:** JWT, 15 minutes expiry, httpOnly cookie
- **Refresh Token:** Random string, 30 days expiry, httpOnly cookie
- **Token Rotation:** Setiap refresh menghasilkan token baru, old token di-revoke
- **Logout:** Clear cookies + revoke refresh token di database

Security Best Practices:

- **httpOnly Cookies:** Token tidak accessible via JavaScript (XSS protection)
- **Secure Flag:** Cookies only sent over HTTPS
- **SameSite=Strict:** CSRF protection
- **Token Rotation:** Mitigasi token theft

6.4 Google Analytics 4 (GA4)

Integration Approach:

1. **Client-side:** Google Tag Manager (GTM) di `<head>`
2. **Server-side:** Measurement Protocol API untuk server events

GTM Setup (dari `src/app/layout.tsx`):

```
<Script
  id="gtm"
  strategy="afterInteractive"
  dangerouslySetInnerHTML={{
```



```

    _html: `
      (function(w,d,s,l,i){w[l]=w[l]||[];w[l].push({'gtm.start':
        new Date().getTime(),event:'gtm.js'});var f=d.getElementsByTagName(s)[0],
        j=d.createElement(s),dl=!!~d.body.className.indexOf('dataLayer')?'&l='+l:'';j.async=true;j.src=
        'https://www.googletagmanager.com/gtm.js?id='+i+dl;f.parentNode.insertBefore(j,f);
      })(window,document,'script','dataLayer','${process.env.NEXT_PUBLIC_GTM_ID}');
    `,
  },
}
/>

```

Custom Events:

```

// Client-side tracking via dataLayer
window.dataLayer = window.dataLayer || [];
window.dataLayer.push({
  event: "article_view",
  article_id: article._id,
  article_title: article.title,
  article_category: article.category.slug,
  article_author: article.author.name,
  article_format: article.format,
});

```

Server-side Tracking (Measurement Protocol):

```

// src/lib/measurement-protocol.ts
import axios from "axios";

export async function trackEvent({
  clientId,
  userId,
  eventName,
  params,
}): {
  clientId: string;
  userId?: string;
  eventName: string;
  params: Record<string, any>;
}) {
  const measurementId = process.env.NEXT_PUBLIC_GA_MEASUREMENT_ID;
  const apiSecret = process.env.GA_MP_API_SECRET;

  if (!measurementId || !apiSecret) return;

  const endpoint =
    process.env.GA_MP_DEBUG === "true"
      ? "https://www.google-analytics.com/debug/mp/collect"
      : "https://www.google-analytics.com/mp/collect";

  await axios.post(
    `${endpoint}?measurement_id=${measurementId}&api_secret=${apiSecret}`,
    {
      client_id: clientId,
      user_id: userId,
      events: [
        {
          name: eventName,
          params: {
            ...params,
            engagement_time_msec: 100,
          },
        },
      ],
    },
  );
}

```

```

}

// Usage
await trackEvent({
  clientId: sessionId,
  userId: user?._id,
  eventName: "article_view",
  params: {
    article_id: article._id,
    article_title: article.title,
    category: article.category.slug,
    author: article.author.name,
  },
});

```

Custom Dimensions & Metrics (via GA Admin API):

```

// scripts/register-ga-custom-definitions.ts
import { AnalyticsAdminServiceClient } from "@google-analytics/admin";

const client = new AnalyticsAdminServiceClient();

// Register custom dimension
await client.createCustomDimension({
  parent: `properties/${propertyId}`,
  customDimension: {
    parameterName: "article_author",
    displayName: "Article Author",
    description: "Author of the article",
    scope: "EVENT",
  },
});

// Register custom metric
await client.createCustomMetric({
  parent: `properties/${propertyId}`,
  customMetric: {
    parameterName: "article_word_count",
    displayName: "Article Word Count",
    description: "Number of words in article",
    measurementUnit: "STANDARD",
    scope: "EVENT",
  },
});

```

Event Tracking List:

- `page_view` - Pageview (auto)
- `article_view` - Article detail view
- `article_share` - Article shared
- `search` - Search query
- `ad_click` - Ad clicked
- `category_view` - Category page view
- `author_view` - Author profile view
- `newsletter_subscribe` - Newsletter subscription (future)

Reporting API (untuk internal analytics dashboard):

```

import { BetaAnalyticsDataClient } from "@google-analytics/data";

const analyticsDataClient = new BetaAnalyticsDataClient();

export async function getArticlePerformance({
  startDate,

```

```
    endDate,
    articleId,
  }: {
    startDate: string;
    endDate: string;
    articleId: string;
  }) {
    const [response] = await analyticsDataClient.runReport({
      property: `properties/${propertyId}`,
      dateRanges: [{ startDate, endDate }],
      dimensions: [{ name: "eventName" }],
      metrics: [
        { name: "eventCount" },
        { name: "activeUsers" },
        { name: "engagementRate" },
      ],
      dimensionFilter: {
        filter: {
          fieldName: "customEvent:article_id",
          stringFilter: { value: articleId },
        },
      },
    });

    return response;
  }
}
```

6.5 Image & Video Processing

Server-side Image Processing (Sharp)

Installation:

```
npm install sharp --save
```

Image Pipeline:

```
// src/lib/image/processImage.ts
import sharp from "sharp";

export async function processImage({
  inputBuffer,
  options,
}): {
  inputBuffer: Buffer;
  options: {
    width?: number;
    height?: number;
    quality?: number;
    format?: "webp" | "jpeg" | "png";
    watermark?: boolean;
  };
} {
  let pipeline = sharp(inputBuffer);

  // Resize
  if (options.width || options.height) {
    pipeline = pipeline.resize(options.width, options.height, {
      fit: "inside",
      withoutEnlargement: true,
    });
  }
}
```

```
// Add watermark
if (options.watermark) {
  const watermarkBuffer = await sharp("public/watermark.png")
    .resize({ width: 200 })
    .toBuffer();

  pipeline = pipeline.composite([
    {
      input: watermarkBuffer,
      gravity: "southeast",
      blend: "over",
    },
  ]);
}

// Convert format & compress
const format = options.format || "webp";
const quality = options.quality || 85;

if (format === "webp") {
  pipeline = pipeline.webp({ quality });
} else if (format === "jpeg") {
  pipeline = pipeline.jpeg({ quality, progressive: true });
} else if (format === "png") {
  pipeline = pipeline.png({ quality, compressionLevel: 9 });
}

return pipeline.toBuffer();
}

// Usage
const optimizedBuffer = await processImage({
  inputBuffer: originalImageBuffer,
  options: {
    width: 1200,
    quality: 85,
    format: "webp",
    watermark: true,
  },
});
```

Multiple Size Variants:

```
const sizes = [
  { name: "thumb", width: 300, height: 200 },
  { name: "medium", width: 800, height: 600 },
  { name: "large", width: 1200, height: 900 },
  { name: "original", width: 1920, height: 1080 },
];

const variants = await Promise.all(
  sizes.map(async (size) => {
    const buffer = await processImage({
      inputBuffer: originalImageBuffer,
      options: {
        width: size.width,
        height: size.height,
        quality: 85,
        format: "webp",
      },
    });

    const key = `articles/${articleId}/images/${ulid()}-${size.name}.webp`;
    await uploadToS3(key, buffer);
    return { size: size.name, key };
  }),
);
```

Client-side Image Resize (Pica)

Installation:

```
npm install pica --save
```

Usage (sebelum upload ke S3):

```
// src/hooks/use-image-upload.ts
import pica from "pica";

export async function resizeImageOnClient(
  file: File,
  maxWidth: number,
  maxHeight: number,
): Promise<Blob> {
  const img = await createImageBitmap(file);

  const canvas = document.createElement("canvas");
  const aspectRatio = img.width / img.height;

  if (img.width > maxWidth) {
    canvas.width = maxWidth;
    canvas.height = maxWidth / aspectRatio;
  } else if (img.height > maxHeight) {
    canvas.height = maxHeight;
    canvas.width = maxHeight * aspectRatio;
  } else {
    canvas.width = img.width;
    canvas.height = img.height;
  }

  const picaInstance = pica();
  await picaInstance.resize(img, canvas, {
    quality: 3, // High quality
    alpha: true,
  });

  return new Promise((resolve) => {
    canvas.toBlob((blob) => resolve(blob!), "image/webp", 0.85);
  });
}
```

Video Processing (FFmpeg)

Installation:

```
npm install fluent-ffmpeg --save
npm install @types/fluent-ffmpeg --save-dev
```

Thumbnail Extraction:

```
// src/lib/media/extractVideoThumbnail.ts
import ffmpeg from "fluent-ffmpeg";
import { promisify } from "util";
import fs from "fs";

export async function extractVideoThumbnail(
  videoPath: string,
```

```

    outputPath: string,
    timestamp: string = "00:00:03", // 3 seconds in
  ): Promise<void> {
    return new Promise((resolve, reject) => {
      ffmpeg(videoPath)
        .screenshots({
          timestamps: [timestamp],
          filename: outputPath,
          size: "1280x720",
        })
        .on("end", () => resolve())
        .on("error", (err) => reject(err));
    });
  }

// Usage
const videoKey = "articles/abc123/videos/xyz.mp4";
const videoUrl = await getS3Url(videoKey);

const thumbnailPath = "/tmp/thumbnail.jpg";
await extractVideoThumbnail(videoUrl, thumbnailPath);

// Upload thumbnail ke S3
const thumbnailBuffer = await fs.promises.readFile(thumbnailPath);
const thumbnailKey = videoKey.replace(".mp4", "-thumb.jpg");
await uploadToS3(thumbnailKey, thumbnailBuffer);

// Cleanup
await fs.promises.unlink(thumbnailPath);

```

Video Metadata Extraction:

```

import ffmpeg from "fluent-ffmpeg";

export async function getVideoMetadata(videoPath: string): Promise<{
  duration: number;
  width: number;
  height: number;
  codec: string;
  bitrate: number;
}> {
  return new Promise((resolve, reject) => {
    ffmpeg.ffprobe(videoPath, (err, metadata) => {
      if (err) return reject(err);

      const videoStream = metadata.streams.find(
        (s) => s.codec_type === "video",
      );
      if (!videoStream) return reject(new Error("No video stream found"));

      resolve({
        duration: metadata.format.duration || 0,
        width: videoStream.width || 0,
        height: videoStream.height || 0,
        codec: videoStream.codec_name || "",
        bitrate: metadata.format.bit_rate || 0,
      });
    });
  });
}

```

Optimization Pipeline:

1. **Client-side:** Resize image dengan Pica sebelum upload (mengurangi bandwidth)
2. **Server-side:** Process dengan Sharp (watermark, multiple variants, WebP conversion)

3. **Storage:** Upload ke S3 dengan cache headers
4. **Delivery:** Serve via CDN dengan edge caching

7. Frontend Architecture

7.1 Next.js App Router

Route Groups:

(public) - **Public Routes** (no authentication required):

```
src/app/(public)/
├── layout.tsx                # Navbar + Footer layout
├── page.tsx                  # Homepage
├── HomePageClient.tsx        # Client component
├── [category]/[yyyy]/[mm]/[dd]/[slug]/
│   └── page.tsx              # Article detail (structured URL)
│                               # Server Component, ISR enabled
├── category/[category]/
│   ├── page.tsx              # Category listing
│   └── CategoryClient.tsx
├── indeks/                   # All articles index
│   ├── page.tsx
│   └── NewsIndeksClient.tsx
├── news/[...segments]/
│   ├── page.tsx              # Legacy & structured URL handler
│   └──                      # Catch-all route
├── penulis/[slug]/
│   ├── page.tsx              # Author profile
│   └── AuthorClient.tsx
├── search/                   # Search results
│   ├── page.tsx
│   └── SearchClient.tsx
```

(inside) - **Static Info Pages:**

```
src/app/(inside)/
├── layout.tsx                # Custom layout
├── about-us/                  # Tentang kami
│   ├── page.tsx
│   ├── AboutUsClient.tsx
│   └── MouseBouncing.tsx    # Animation component
├── disclaimer/               # Disclaimer
│   └── page.tsx
├── pedoman-media-siber/
│   └── page.tsx             # Pedoman jurnalistik
```

admin - **CMS Admin Panel** (authentication + role check):

```
src/app/admin/
├── (dashboard)/
│   ├── layout.tsx           # Sidebar + Header
│   ├── page.tsx             # Dashboard home
│   ├── articles/            # Article management
│   ├── categories/          # Category CRUD
│   ├── users/               # User management
│   ├── analytics/           # Analytics dashboard
│   ├── ads/                 # Ads management
│   └── [other admin routes]/
└── create-article/          # Create article (separate layout)
    └── page.tsx
```

Dynamic Routes:

- `[category]/[yyyy]/[mm]/[dd]/[slug]` - Structured article URL
- `news/[...segments]` - Catch-all untuk legacy `/news/{slug}` dan structured URL
- `[id]` - Dynamic ID routes (edit artikel, user detail, dll)

Parallel Routes: Not used (belum diimplementasikan)

Intercepting Routes: Not used (belum diimplementasikan)

Route Protection:

```
// src/app/admin/(dashboard)/layout.tsx
export default async function DashboardLayout({
  children,
}): {
  children: React.ReactNode;
}) {
  const user = await getUserFromCookies();

  if (!user) {
    redirect('/api/auth/login');
  }

  // Check if user has admin access
  const adminRoles = ['admin', 'editor-in-chief', 'managing-editor', 'head-of', 'editor', 'writer',
'reporter'];
  if (!adminRoles.includes(user.role)) {
    return <div>Access Denied</div>;
  }

  return (
    <div className="flex">
      <Sidebar user={user} />
      <main className="flex-1">{children}</main>
    </div>
  );
}
```

7.2 Data Fetching

Server Components Data Loading (default):

```
// src/app/(public)/page.tsx
export default async function HomePage() {
  // Direct MongoDB query (no API call overhead)
  const articles = await getCollection('articles')
    .find({ status: 'PUBLISHED' })
    .sort({ publishedAt: -1 })
    .limit(10)
    .toArray();

  const categories = await getCollection('categories')
    .find({ featured: true })
    .sort({ featuredOrder: 1 })
    .toArray();

  return (
    <div>
      <FeaturedSection articles={articles} />
      <CategoriesGrid categories={categories} />
    </div>
  );
}
```


Client-side Fetching (React Query):

```
// src/hooks/use-articles.ts
import { useQuery } from '@tanstack/react-query';

export function useArticles(params: {
  status?: string;
  categorySlug?: string;
  limit?: number;
}) {
  return useQuery({
    queryKey: ['articles', params],
    queryFn: async () => {
      const query = new URLSearchParams(params as any);
      const res = await fetch(`/api/articles?${query}`);
      if (!res.ok) throw new Error('Failed to fetch articles');
      return res.json();
    },
    staleTime: 5 * 60 * 1000, // 5 minutes
    refetchOnWindowFocus: false,
  });
}

// Usage in component
'use client';

export function ArticleList() {
  const { data, isLoading, error } = useArticles({
    status: 'PUBLISHED',
    limit: 10,
  });

  if (isLoading) return <Skeleton />;
  if (error) return <ErrorComponent />;

  return (
    <div>
      {data.data.map((article) => (
        <ArticleCard key={article._id} article={article} />
      ))}
    </div>
  );
}
```

Client-side Fetching (SWR):

```
// src/hooks/use-notifications.ts
import useSWR from "swr";

const fetcher = (url: string) => fetch(url).then((r) => r.json());

export function useNotifications() {
  const { data, error, mutate } = useSWR("/api/notification", fetcher, {
    refreshInterval: 30000, // Poll every 30s
    revalidateOnFocus: true,
  });

  return {
    notifications: data?.data || [],
    isLoading: !error && !data,
    isError: error,
    refresh: mutate,
  };
}
```

Caching Strategy:

1. **Server Components:** Fetch data saat render, cached by Next.js (default)
2. **ISR (Incremental Static Regeneration):** Article pages dengan revalidation

```
// src/app/(public)/[category]/[yyyy]/[mm]/[dd]/[slug]/page.tsx
export const revalidate = 3600; // 1 hour
```

3. **React Query:** Client-side cache dengan `staleTime` dan `cacheTime`
4. **SWR:** Client-side cache dengan `dedupingInterval`

Revalidation:

- **Time-based:** ISR revalidate after N seconds
- **On-demand:** `revalidatePath()` atau `revalidateTag()` setelah mutation
- **Manual:** `mutate()` dari React Query/SWR

7.3 State Management

Server State (React Query):

```
// src/app/admin/(dashboard)/articles/page.tsx
'use client';

import { QueryClient, QueryClientProvider } from '@tanstack/react-query';

const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      staleTime: 60 * 1000,
      cacheTime: 5 * 60 * 1000,
      refetchOnWindowFocus: false,
    },
  },
});

export default function ArticlesPage() {
  return (
    <QueryClientProvider client={queryClient}>
      <ArticleList />
    </QueryClientProvider>
  );
}
```

Client State (React hooks):

```
// src/components/admin/articles/ArticleFilters.tsx
'use client';

import { useState } from 'react';

export function ArticleFilters({ onFilterChange }: { onFilterChange: (filters: any) => void }) {
  const [status, setStatus] = useState<string>('');
  const [category, setCategory] = useState<string>('');

  const handleApply = () => {
    onFilterChange({ status, category });
  };

  return (
    <div className="flex gap-4">
      <Select value={status} onChange={setStatus}>

```

```

    <option value="">All Status</option>
    <option value="DRAFT">Draft</option>
    <option value="PUBLISHED">Published</option>
  </Select>
  <Button onClick={handleApply}>Apply</Button>
</div>
);
}

```

Form State (React Hook Form):

```

// src/components/admin/articles/ArticleForm.tsx
'use client';

import { useForm } from 'react-hook-form';
import { zodResolver } from '@hookform/resolvers/zod';
import { z } from 'zod';

const articleSchema = z.object({
  title: z.string().min(10).max(200),
  excerpt: z.string().min(50).max(500),
  content: z.string().min(100),
  categoryId: z.string(),
  tags: z.array(z.string()),
});

type ArticleFormData = z.infer<typeof articleSchema>;

export function ArticleForm({ article }: { article?: Article }) {
  const {
    register,
    handleSubmit,
    formState: { errors },
  } = useForm<ArticleFormData>({
    resolver: zodResolver(articleSchema),
    defaultValues: article,
  });

  const onSubmit = async (data: ArticleFormData) => {
    const res = await fetch('/api/articles', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify(data),
    });

    if (res.ok) {
      toast.success('Article saved');
    }
  };

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <input {...register('title')} placeholder="Title" />
      {errors.title && <span>{errors.title.message}</span>}

      <textarea {...register('content')} placeholder="Content" />
      {errors.content && <span>{errors.content.message}</span>}

      <button type="submit">Save</button>
    </form>
  );
}

```

URL State (searchParams):

```
// src/app/(public)/search/page.tsx
export default function SearchPage({
  searchParams,
}): {
  searchParams: { q?: string; category?: string };
} {
  const query = searchParams.q || '';
  const category = searchParams.category || '';

  // Fetch based on URL params
  const results = await searchArticles({ query, category });

  return (
    <div>
      <SearchInput defaultValue={query} />
      <SearchResults results={results} />
    </div>
  );
}

// src/components/search/SearchInput.tsx
'use client';

import { useRouter, useSearchParams } from 'next/navigation';

export function SearchInput({ defaultValue }: { defaultValue: string }) {
  const router = useRouter();
  const searchParams = useSearchParams();

  const handleSearch = (query: string) => {
    const params = new URLSearchParams(searchParams.toString());
    params.set('q', query);
    router.push(`/search?${params.toString()}`);
  };

  return <input defaultValue={defaultValue} onChange={(e) => handleSearch(e.target.value)} />;
}
```

7.4 Component Architecture

shadcn/ui Design System:

- **Installation:** `npx shadcn@latest add button`
- **Components:** 40+ components di `src/components/ui/`
- **Customization:** Via `tailwind.config.ts` dan CSS variables

Component Structure:

```
// src/components/ui/button.tsx
import * as React from 'react';
import { cva, type VariantProps } from 'class-variance-authority';
import { cn } from '@lib/utlis';

const buttonVariants = cva(
  'inline-flex items-center justify-center rounded-md text-sm font-medium transition-colors',
  {
    variants: {
      variant: {
        default: 'bg-primary text-primary-foreground hover:bg-primary/90',
        destructive: 'bg-destructive text-destructive-foreground hover:bg-destructive/90',
        outline: 'border border-input hover:bg-accent hover:text-accent-foreground',
        ghost: 'hover:bg-accent hover:text-accent-foreground',
      },
      size: {
        default: 'h-10 px-4 py-2',

```

```

        sm: 'h-9 rounded-md px-3',
        lg: 'h-11 rounded-md px-8',
        icon: 'h-10 w-10',
      },
    },
    defaultVariants: {
      variant: 'default',
      size: 'default',
    },
  }
}
);

export interface ButtonProps
  extends React.ButtonHTMLAttributes<HTMLButtonElement>,
    VariantProps<typeof buttonVariants> {}

const Button = React.forwardRef<HTMLButtonElement, ButtonProps>(
  ({ className, variant, size, ...props }, ref) => {
    return (
      <button
        className={cn(buttonVariants({ variant, size, className }))}
        ref={ref}
        {...props}
      />
    );
  }
);

export { Button, buttonVariants };

```

Compound Components Pattern:

```

// src/components/ui/card.tsx
export function Card({ children, className }: { children: React.ReactNode; className?: string }) {
  return <div className={cn('rounded-lg border bg-card', className)}>{children}</div>;
}

export function CardHeader({ children }: { children: React.ReactNode }) {
  return <div className="p-6">{children}</div>;
}

export function CardTitle({ children }: { children: React.ReactNode }) {
  return <h3 className="text-2xl font-semibold">{children}</h3>;
}

export function CardContent({ children }: { children: React.ReactNode }) {
  return <div className="p-6 pt-0">{children}</div>;
}

// Usage
<Card>
  <CardHeader>
    <CardTitle>Article Stats</CardTitle>
  </CardHeader>
  <CardContent>
    <p>Total views: 1234</p>
  </CardContent>
</Card>

```

Composition over Inheritance:

```

// src/components/articles/ArticleCard.tsx
import { Card, CardHeader, CardTitle, CardContent } from '@components/ui/card';
import { Badge } from '@components/ui/badge';
import { Avatar } from '@components/ui/avatar';

```

```
export function ArticleCard({ article }: { article: Article }) {
  return (
    <Card>
      <CardHeader>
        <Badge>{article.category.name}</Badge>
        <CardTitle>{article.title}</CardTitle>
      </CardHeader>
      <CardContent>
        <p>{article.excerpt}</p>
        <div className="flex items-center gap-2 mt-4">
          <Avatar src={article.author.avatar} />
          <span>{article.author.name}</span>
        </div>
      </CardContent>
    </Card>
  );
}
```

Component Props Strategy:

- **Controlled vs Uncontrolled:** Form inputs controlled via React Hook Form
- **Prop Drilling:** Minimize via composition dan context
- **Render Props:** Untuk komponen dengan custom render logic
- **Children as Function:** Untuk advanced composition

7.5 Rich Text Editor (TipTap)

Editor Configuration (dari `src/components/typetap/Editor.tsx`):

```
'use client';

import { useEditor, EditorContent } from '@tiptap/react';
import StarterKit from '@tiptap/starter-kit';
import Image from '@tiptap/extension-image';
import Link from '@tiptap/extension-link';
import Underline from '@tiptap/extension-underline';
import TextAlign from '@tiptap/extension-text-align';
import Highlight from '@tiptap/extension-highlight';
import Youtube from '@tiptap/extension-youtube';
import Placeholder from '@tiptap/extension-placeholder';

export function TipTapEditor({ content, onChange }: {
  content: string;
  onChange: (html: string) => void;
}) {
  const editor = useEditor({
    extensions: [
      StarterKit.configure({
        heading: {
          levels: [1, 2, 3],
        },
      }),
      Image.configure({
        HTMLAttributes: {
          class: 'max-w-full h-auto',
        },
      }),
      Link.configure({
        openOnClick: false,
        HTMLAttributes: {
          class: 'text-blue-600 underline',
        },
      }),
      Underline,
    ],
    content,
    onChange,
  });
}
```

```

    TextAlign.configure({
      types: ['heading', 'paragraph'],
      alignments: ['left', 'center', 'right', 'justify'],
    }),
    Highlight.configure({
      multicolor: true,
    }),
    Youtube.configure({
      width: 640,
      height: 480,
    }),
    Placeholder.configure({
      placeholder: 'Tulis artikel Anda di sini...',
    }),
  ],
  content,
  onUpdate: ({ editor }) => {
    onChange(editor.getHTML());
  },
  editorProps: {
    attributes: {
      class: 'prose max-w-none focus:outline-none min-h-[400px] p-4',
    },
  },
});

return (
  <div className="border rounded-lg">
    <EditorToolbar editor={editor} />
    <EditorContent editor={editor} />
  </div>
);
}

```

Custom Extensions:

```

// src/lib/tiptap/ImageUploadExtension.ts
import { Node } from "@tiptap/core";
import { Plugin, PluginKey } from "@tiptap/pm/state";

export const ImageUpload = Node.create({
  name: "imageUpload",

  addProseMirrorPlugins() {
    return [
      new Plugin({
        key: new PluginKey("imageUploadPlugin"),
        props: {
          handlePaste(view, event) {
            const items = Array.from(event.clipboardData?.items || []);
            const imageItem = items.find((item) =>
              item.type.startsWith("image/"),
            );

            if (imageItem) {
              const file = imageItem.getAsFile();
              if (file) {
                // Upload image
                uploadImageAndInsert(file, view);
                return true;
              }
            }
            return false;
          },
          handleDrop(view, event) {
            const files = Array.from(event.dataTransfer?.files || []);
            const imageFile = files.find((file) =>

```

```

        file.type.startsWith("image/"),
      );

      if (imageFile) {
        uploadImageAndInsert(imageFile, view);
        return true;
      }
      return false;
    },
  ),
}),
]);
});

async function uploadImageAndInsert(file: File, view: any) {
  // 1. Resize dengan Pica
  const resizedBlob = await resizeImageOnClient(file, 1200, 900);

  // 2. Get presigned URL
  const { url, key } = await fetch("/api/media/presigned-url", {
    method: "POST",
    body: JSON.stringify({
      filename: file.name,
      mimetype: "image/webp",
      size: resizedBlob.size,
    }),
  }).then((r) => r.json());

  // 3. Upload to S3
  await fetch(url, {
    method: "PUT",
    body: resizedBlob,
    headers: { "Content-Type": "image/webp" },
  });

  // 4. Save metadata
  const media = await fetch("/api/media", {
    method: "POST",
    body: JSON.stringify({ key, caption: "", credit: "" }),
  }).then((r) => r.json());

  // 5. Insert to editor
  const imageUrl = media.data.url;
  const { state, dispatch } = view;
  const node = state.schema.nodes.image.create({ src: imageUrl });
  const transaction = state.tr.replaceSelectionWith(node);
  dispatch(transaction);
}

```

Content Serialization:

- **Storage Format:** HTML string di MongoDB `articles.content`
- **Parsing:** `html-react-parser` untuk render di frontend
- **Sanitization:** TipTap built-in XSS protection

Media Embedding:

- **Images:** Upload via drag-drop, paste, atau toolbar button
- **YouTube:** Via custom toolbar button dengan URL input
- **Twitter/X:** Via `react-tweet` component (inserted as custom node)
- **Instagram/TikTok:** Via `react-social-media-embed`

8. Media Management

8.1 Upload Pipeline

Complete Flow:

1. Client select file (drag-drop, file input, clipboard paste)
2. Client-side validation (type, size)
3. Client-side resize dengan Pica (max 1920px width)
4. Request presigned URL dari `/api/media/presigned-url`
5. Upload langsung ke S3 via presigned URL (PUT)
6. Confirm upload ke `/api/media` dengan key & metadata
7. Server process dengan Sharp (thumbnails, watermark, WebP conversion)
8. Upload variants ke S3
9. Save metadata ke MongoDB
10. Return media object dengan CDN URL

File Size Limits:

- **Images:** Max 10 MB (before resize)
- **Videos:** Max 100 MB
- **After resize:** Typically < 2 MB (WebP, quality 85%)

Allowed MIME Types:

- **Images:** `image/jpeg`, `image/png`, `image/webp`, `image/gif`
- **Videos:** `video/mp4`, `video/webm`

8.2 Image Processing Flow

Client-side Preview & Crop:

```
// src/components/media/ImageCropper.tsx
'use client';

import ReactCrop, { type Crop } from 'react-image-crop';
import 'react-image-crop/dist/ReactCrop.css';

export function ImageCropper({ src, onCropComplete }: {
  src: string;
  onCropComplete: (croppedBlob: Blob) => void;
}) {
  const [crop, setCrop] = useState<Crop>();
  const imageRef = useRef<HTMLImageElement>(null);

  const handleCropComplete = async () => {
    if (!crop || !imageRef.current) return;

    const canvas = document.createElement('canvas');
    const scaleX = imageRef.current.naturalWidth / imageRef.current.width;
    const scaleY = imageRef.current.naturalHeight / imageRef.current.height;

    canvas.width = crop.width * scaleX;
    canvas.height = crop.height * scaleY;

    const ctx = canvas.getContext('2d');
    ctx?.drawImage(
      imageRef.current,
      crop.x * scaleX,
      crop.y * scaleY,
      crop.width * scaleX,
      crop.height * scaleY,
      0,
      0,
      canvas.width,
      canvas.height
    );

    canvas.toBlob((blob) => {
```

```
    if (blob) onCropComplete(blob);
  }, 'image/webp', 0.85);
};

return (
  <div>
    <ReactCrop crop={crop} onChange={setCrop}>
      <img ref={imageRef} src={src} alt="Crop preview" />
    </ReactCrop>
    <button onClick={handleCropComplete}>Apply Crop</button>
  </div>
);
}
```

Server-side Optimization (Sharp):

```
// Sudah dijelaskan di bagian 6.5 (Infrastruktur)
// Multiple size variants: thumb (300x200), medium (800x600), large (1200x900), original (1920x1080)
// Watermark placement: southeast corner
// WebP conversion: quality 85%
```

WebP Conversion Benefits:

- **Size Reduction:** 25-35% smaller than JPEG at same quality
- **Browser Support:** 95%+ (fallback to JPEG jika perlu)
- **Lossless & Lossy:** Support both modes

Featured Image Handling:

- **Storage:** `articles.featuredImage.mediaId` reference ke `media._id`
- **Denormalization:** `articles.featuredImage.url` (CDN URL) disimpan untuk performa
- **Variants:** Serve responsive sizes via `<Image>` component
- **Caption & Credit:** Per-article attribution (bukan dari media collection)

8.3 Video Processing

Thumbnail Extraction (FFmpeg):

```
// Sudah dijelaskan di bagian 6.5 (Infrastruktur)
// Extract frame at 3 seconds
// Size: 1280x720
// Format: JPEG
// Upload thumbnail ke S3: {video-key}-thumb.jpg
```

Format Validation:

```
export async function validateVideo(
  file: File,
): Promise<{ valid: boolean; error?: string }> {
  const allowedTypes = ["video/mp4", "video/webm"];

  if (!allowedTypes.includes(file.type)) {
    return { valid: false, error: "Only MP4 and WebM formats allowed" };
  }

  if (file.size > 100 * 1024 * 1024) {
    // 100 MB
    return { valid: false, error: "File size exceeds 100 MB" };
  }
}
```

```
    return { valid: true };  
  }
```

Size Limits:

- **Max File Size:** 100 MB
- **Recommended:** < 50 MB untuk performa streaming optimal
- **Compression:** Client-side compression not implemented (use server-side tools)

CDN Delivery:

- **S3 Static Hosting:** Video served langsung dari S3
- **CloudFront:** Optional untuk CDN caching
- **Streaming:** No adaptive bitrate (future enhancement: HLS/DASH)

8.4 Media Storage Structure

S3 Bucket Organization:

```
s3://arasvara-media/  
├── articles/  
│   ├── {article-id}/  
│   │   ├── images/  
│   │   │   ├── {ulid}.webp           # Original  
│   │   │   ├── {ulid}-thumb.webp    # 300x200  
│   │   │   ├── {ulid}-medium.webp   # 800x600  
│   │   │   └── {ulid}-large.webp    # 1200x900  
│   │   └── videos/  
│   │       ├── {ulid}.mp4           # Original video  
│   │       └── {ulid}-thumb.jpg      # Video thumbnail  
│   └── ...  
├── avatars/  
│   └── {user-id}.webp               # User avatar  
├── configuration/  
│   └── {config-key}.{ext}           # Site config files (logo, favicon, etc)  
├── ads/  
│   └── {ad-id}.{ext}                # Ad banners  
└── temp/  
    └── {ulid}.{ext}                 # Temporary uploads (cleaned after 24h)
```

File Naming Convention:

- **ULID:** Lexicographically sortable unique ID (better than UUID)
- **Extension:** `.webp` untuk images, `.mp4` untuk videos
- **Suffix:** `-thumb`, `-medium`, `-large` untuk variants

Cache Headers (dari S3 upload):

```
{  
  CacheControl: 'public, max-age=31536000, immutable', // 1 year  
  ContentType: 'image/webp',  
}
```

Lifecycle Policy (S3):

- **temp/ folder:** Auto-delete after 1 day
- **Orphaned media:** Manual cleanup via `/api/media/cleanup` (admin cron job)
- **Deleted articles:** Media retained untuk audit (soft delete)

Storage Costs (estimated):

- **100,000 articles** × 5 images × 4 variants × 200 KB = ~400 GB

- **S3 Standard:** \$0.023/GB/month = ~\$9.20/month
 - **Transfer:** First 100 GB/month free, then \$0.09/GB
-

9. Search & Indexing

9.1 Search Implementation

MongoDB Text Index:

```
// Create text index (one-time setup)
db.articles.createIndex(
  {
    title: "text",
    excerpt: "text",
    content: "text",
  },
  {
    weights: {
      title: 10, // Title most important
      excerpt: 5, // Excerpt medium importance
      content: 1, // Content least important
    },
    name: "articles_text_search",
  },
);
```

Search Query Construction:

```
// src/services/searchService.ts
export async function searchArticles({
  query,
  category,
  limit = 10,
  cursor,
}): {
  query: string;
  category?: string;
  limit?: number;
  cursor?: string;
}) {
  const collection = await getCollection("articles");

  const filter: any = {
    status: "PUBLISHED",
    $text: { $search: query },
  };

  if (category) {
    filter["category.slug"] = category;
  }

  if (cursor) {
    const [score, id] = decodeCursor(cursor);
    filter.$or = [
      { score: { $lt: score } },
      { score: { $eq: score }, _id: { $lt: new ObjectId(id) } },
    ];
  }

  const results = await collection
    .find(filter, {
      projection: {
        content: 0, // Exclude heavy field
        score: { $meta: "textScore" },
      },
    })
    .limit(limit)
    .sort({ score: -1 })
    .toArray();

  return {
    query,
    category,
    limit,
    cursor,
    results,
  };
}
```

```
    },
  })
  .sort({ score: { $meta: "textScore" }, _id: -1 })
  .limit(limit + 1)
  .toArray();

const hasMore = results.length > limit;
const items = hasMore ? results.slice(0, limit) : results;
const nextCursor = hasMore
  ? encodeCursor(items[items.length - 1].score, items[items.length - 1]._id)
  : null;

return { items, nextCursor };
}
```

Relevance Scoring:

- **Text Score:** MongoDB `$meta: 'textScore'` berdasarkan TF-IDF
- **Weights:** Title (10x), Excerpt (5x), Content (1x)
- **Sorting:** By score descending, then by `_id` (tie-breaker)

Fuzzy Matching:

- **MongoDB Text Search:** Support stemming & stop words (English default)
- **Limitations:** No typo tolerance, no phonetic matching
- **Future Enhancement:** Algolia, Elasticsearch, atau Typesense untuk advanced search

9.2 Search Filters

Available Filters:

1. **Category:** Filter by `category.slug`
2. **Date Range:** Filter by `publishedAt` (start date, end date)
3. **Author:** Filter by `authorId`
4. **Format:** Filter by `format` (STANDARD | GALLERY)

Filter Implementation:

```
export async function searchArticles({
  query,
  category,
  authorId,
  format,
  startDate,
  endDate,
}: SearchParams) {
  const filter: any = {
    status: "PUBLISHED",
    $text: { $search: query },
  };

  if (category) {
    filter["category.slug"] = category;
  }

  if (authorId) {
    filter.authorId = new ObjectId(authorId);
  }

  if (format) {
    filter.format = format;
  }

  if (startDate || endDate) {
    filter.publishedAt = {};
  }
}
```

```

    if (startDate) filter.publishedAt.$gte = new Date(startDate);
    if (endDate) filter.publishedAt.$lte = new Date(endDate);
  }

  // ... rest of search logic
}

```

Filter UI (Client Component):

```

'use client';

export function SearchFilters({ onFilterChange }: {
  onFilterChange: (filters: SearchFilters) => void;
}) {
  const [category, setCategory] = useState('');
  const [dateRange, setDateRange] = useState<DateRange>();

  const handleApply = () => {
    onFilterChange({
      category,
      startDate: dateRange?.from,
      endDate: dateRange?.to,
    });
  };

  return (
    <div className="space-y-4">
      <Select value={category} onChange={setCategory}>
        <SelectTrigger>Select Category</SelectTrigger>
        <SelectContent>
          <SelectItem value="">All</SelectItem>
          {categories.map(c => (
            <SelectItem key={c._id} value={c.slug}>{c.name}</SelectItem>
          ))}
        </SelectContent>
      </Select>

      <DateRangePicker value={dateRange} onChange={setDateRange} />

      <Button onClick={handleApply}>Apply Filters</Button>
    </div>
  );
}

```

9.3 Pagination Strategy

Cursor-based Pagination (untuk search results):

```

// Encode cursor
function encodeCursor(score: number, id: string): string {
  return Buffer.from(`${score}:${id}`).toString("base64url");
}

// Decode cursor
function decodeCursor(cursor: string): [number, string] {
  const [score, id] = Buffer.from(cursor, "base64url").toString().split(":");
  return [parseFloat(score), id];
}

// Usage
const { items, nextCursor } = await searchArticles({
  query: "teknologi",
  limit: 10,
  cursor: request.query.cursor,
});

```

```
});  
  
return {  
  data: items,  
  meta: { nextCursor },  
};
```

Page Size Limits:

- **Default:** 10 items per page
- **Max:** 100 items per page
- **Recommendation:** 20-50 untuk optimal UX

Total Count Calculation:

```
// For search results, total count is expensive (full text scan)  
// Option 1: Estimate via $count (acceptable for pagination UI)  
const totalCount = await collection.countDocuments(filter);  
  
// Option 2: Skip total count, use "Load More" instead of page numbers  
// More performant untuk large datasets
```

10. Analytics & Reporting

10.1 GA4 Integration

Custom Events (sudah dijelaskan di bagian 6.4):

- `page_view` (auto)
- `article_view`
- `article_share`
- `search`
- `ad_click`
- `category_view`
- `author_view`

Custom Dimensions:

- `article_id` (event-scoped)
- `article_title` (event-scoped)
- `article_category` (event-scoped)
- `article_author` (event-scoped)
- `article_format` (event-scoped)
- `user_role` (user-scoped, untuk authenticated users)

Custom Metrics:

- `article_word_count` (event-scoped)
- `time_to_first_byte` (event-scoped)
- `engagement_time` (event-scoped)

User Properties (for authenticated users):

```
// Set user properties via GTM dataLayer  
window.dataLayer.push({  
  user_id: user._id,  
  user_role: user.role,  
  user_team: user.team?.slug,  
});
```

Event Parameters:

```
// Client-side tracking
window.dataLayer.push({
  event: "article_view",
  article_id: article._id,
  article_title: article.title,
  article_category: article.category.slug,
  article_author: article.author.name,
  article_format: article.format,
  article_word_count: article.content.split(" ").length,
});
```

10.2 Internal Analytics

Page View Tracking:

```
// src/app/api/analytics/pageview/route.ts
export async function POST(request: NextRequest) {
  const { path, sessionId, referrer } = await request.json();

  await getCollection("page_views").insertOne({
    path,
    sessionId,
    ip: request.ip || request.headers.get("x-forwarded-for"),
    userAgent: request.headers.get("user-agent"),
    referrer,
    viewedAt: new Date(),
  });

  return NextResponse.json({ message: "Tracked" });
}
```

Article Performance Metrics:

```
// src/services/analytics/articlePerformanceService.ts
export async function getArticlePerformance(articleId: string) {
  const views = await getCollection("article_views").countDocuments({
    articleId: new ObjectId(articleId),
  });

  const uniqueVisitors = await getCollection("article_views")
    .distinct("sessionId", { articleId: new ObjectId(articleId) })
    .then((arr) => arr.length);

  const avgTimeOnPage = await getCollection("article_views")
    .aggregate([
      { $match: { articleId: new ObjectId(articleId) } },
      { $group: { _id: null, avgTime: { $avg: "$timeOnPage" } } },
    ])
    .toArray()
    .then((r) => r[0]?.avgTime || 0);

  return { views, uniqueVisitors, avgTimeOnPage };
}
```

User Activity Tracking:

```
// Track via editor_activities collection (sudah dijelaskan di bagian 3)
// Actions: CREATE, UPDATE, DELETE, PUBLISH, SCHEDULE, TAKE_DOWN, REJECT, RESTORE
```


KPI Calculation:

```
// src/services/reports/kpiUserService.ts
export async function calculateUserKPI({
  userId,
  startDate,
  endDate,
}): {
  userId: string;
  startDate: Date;
  endDate: Date;
} {
  const articlesPublished = await getCollection("articles").countDocuments({
    authorId: new ObjectId(userId),
    status: "PUBLISHED",
    publishedAt: { $gte: startDate, $lte: endDate },
  });

  const totalViews = await getCollection("article_views")
    .aggregate([
      {
        $lookup: {
          from: "articles",
          localField: "articleId",
          foreignField: "_id",
          as: "article",
        },
      },
      { $unwind: "$article" },
      {
        $match: {
          "article.authorId": new ObjectId(userId),
          viewedAt: { $gte: startDate, $lte: endDate },
        },
      },
      { $count: "total" },
    ])
    .toArray()
    .then((r) => r[0]?.total || 0);

  const avgViewsPerArticle =
    articlesPublished > 0 ? totalViews / articlesPublished : 0;

  return {
    articlesPublished,
    totalViews,
    avgViewsPerArticle,
  };
}
```

10.3 KPI Dashboard**Writer Performance** (dari `memory/KPI.md`):

- **Artikel Diterbitkan:** Count published articles by authorId
- **Total Views:** Sum article_views untuk artikel author
- **Avg Views per Article:** totalViews / articlesPublished
- **Top Articles:** Sort by viewCount descending

Editor Performance (14-day rolling):

- **Artikel Diproses:** Count editor_activities dengan action APPROVE, REJECT, PUBLISH (last 14 days)
- **Avg Processing Time:** Dari PENDING_REVIEW sampai APPROVED/REJECTED
- **Revision Rate:** (REJECTED / TOTAL_REVIEWED) × 100%
- **Publish Rate:** (PUBLISHED / TOTAL_REVIEWED) × 100%

Article Engagement Metrics:

- **Views:** Count dari `article_views`
- **Unique Visitors:** Distinct `sessionId` dari `article_views`
- **Avg Time on Page:** Avg dari `article_views.timeOnPage`
- **Bounce Rate:** $(\text{Single-page sessions} / \text{Total sessions}) \times 100\%$
- **Social Shares:** (Not yet tracked)

Category Performance:

- **Total Articles:** Count articles per category
- **Total Views:** Sum `article_views` per category
- **Avg Views per Article:** `totalViews / totalArticles`
- **Top Categories:** Sort by total views descending

Dashboard UI Components:

```
// src/components/analytics/KPICard.tsx
export function KPICard({ title, value, change, icon }: {
  title: string;
  value: string | number;
  change?: number; // % change from previous period
  icon: React.ReactNode;
}) {
  return (
    <Card>
      <CardHeader className="flex flex-row items-center justify-between">
        <CardTitle className="text-sm font-medium">{title}</CardTitle>
        {icon}
      </CardHeader>
      <CardContent>
        <div className="text-2xl font-bold">{value}</div>
        {change !== undefined && (
          <p className={cn(
            'text-xs',
            change >= 0 ? 'text-green-600' : 'text-red-600'
          )}>
            {change >= 0 ? '+' : ''}{change.toFixed(1)}% from last period
          </p>
        )}
      </CardContent>
    </Card>
  );
}

// Usage
<div className="grid gap-4 md:grid-cols-2 lg:grid-cols-4">
  <KPICard
    title="Total Articles"
    value={stats.totalArticles}
    change={stats.articlesChangePercent}
    icon={<FileText className="h-4 w-4" />}
  />
  <KPICard
    title="Total Views"
    value={stats.totalViews.toLocaleString()}
    change={stats.viewsChangePercent}
    icon={<Eye className="h-4 w-4" />}
  />
  <KPICard
    title="Unique Visitors"
    value={stats.uniqueVisitors.toLocaleString()}
    change={stats.visitorsChangePercent}
    icon={<Users className="h-4 w-4" />}
  />
  <KPICard
    title="Avg Time on Site"
```

```
value={`$${(stats.avgTimeOnSite / 60).toFixed(1)}m`}
  icon={<Clock className="h-4 w-4" />}
/>
</div>
```

11. Scripts & Data Operations

11.1 Migration Scripts

Location: `scripts/migrate-*.ts`

Available Scripts:

1. `migrate-featured-image-filename.ts`

- **Purpose:** Migrate featured image structure
- **What it does:** Update `articles.featuredImage` dari format lama ke format baru
- **Run:** `npm run migrate:featured-image`

2. `migrate-webp-audit.ts`

- **Purpose:** Audit & migrate images to WebP format
- **What it does:** Scan articles, convert JPEG/PNG to WebP, update references
- **Run:** `npm run migrate:webp-audit (dev)`, `npm run migrate:webp-audit:prod (prod)`

3. `migrate-editor-activities-to-audit-log.ts`

- **Purpose:** Migrate `editor_activities` → `audit_log`
- **What it does:** Copy data dari collection lama ke format baru dengan meta enrichment
- **Run:** `npm run migrate:editor-activities`
- **Verification:** `npm run verify:editor-activities-migration`

4. `migrate-structured-path-remove-news-prefix.ts`

- **Purpose:** Remove `/news/` prefix dari structured paths
- **What it does:** Update `articles.publicPath` dari `/news/category/...` ke `/category/...`
- **Run:** `npm run migrate:structured-path-prefix`

5. `upgrade-articles-to-structured-path.ts`

- **Purpose:** Upgrade artikel dari legacy URL ke structured URL
- **What it does:** Generate `publicPath` field berdasarkan `publishedAt` dan `category`
- **Run:** `npm run upgrade:article-paths`

Migration Pattern:

```
// scripts/migrate-example.ts
import { connectToDatabase, getCollection } from "../src/lib/db/db";

async function migrate() {
  await connectToDatabase();
  const collection = await getCollection("articles");

  // 1. Find documents to migrate
  const cursor = collection.find({ needsMigration: true });

  let count = 0;
  for await (const doc of cursor) {
    // 2. Transform data
    const updated = transformDocument(doc);

    // 3. Update document
    await collection.updateOne({ _id: doc._id }, { $set: updated });

    count++;
    if (count % 100 === 0) {
```

```
        console.log(`Migrated ${count} documents...`);
    }
}

console.log(`Migration complete: ${count} documents`);
}

migrate().catch(console.error);
```

11.2 Audit Scripts

Location: `scripts/audit-*.ts`

Available Scripts:

1. `audit-article-titles.ts`

- **Purpose:** Audit article titles untuk quality check
- **Checks:** Length (min 10, max 200 chars), duplicates, special characters
- **Run:** `npm run audit:article-titles`

2. `audit-article-paths.ts`

- **Purpose:** Audit article public paths untuk consistency
- **Checks:** Path format, duplicates, missing paths, broken slugs
- **Run:** `npm run audit:article-paths` (dev), `npm run audit:article-paths:prod` (prod)

3. `audit-article-publish-dates.ts`

- **Purpose:** Audit publish dates untuk anomalies
- **Checks:** Future dates, dates before site launch, timezone issues
- **Run:** `npm run audit:article-publish-dates`

4. `audit-user-names-slugs.ts`

- **Purpose:** Audit user names & slugs untuk conflicts
- **Checks:** Duplicate slugs, invalid slugs, nameNormalized consistency
- **Run:** `npm run audit:user-slugs`

Audit Pattern:

```
// scripts/audit-example.ts
import { getCollection } from "../src/lib/db/db";

async function audit() {
    const collection = await getCollection("articles");

    const issues: Array<{ id: string; issue: string }> = [];

    const cursor = collection.find({});
    for await (const doc of cursor) {
        // Check rules
        if (doc.title.length < 10) {
            issues.push({ id: doc._id, issue: "Title too short" });
        }
        if (!doc.publicPath) {
            issues.push({ id: doc._id, issue: "Missing publicPath" });
        }
    }

    console.log(`Found ${issues.length} issues`);
    console.table(issues);

    // Optionally write to file
```

```
await fs.writeFile("audit-report.json", JSON.stringify(issues, null, 2));
}
```

11.3 Verification Scripts

Location: `scripts/verify-*.ts`

1. `verify-public-media-url.ts`

- **Purpose:** Verify media URLs are accessible
- **Checks:** HTTP 200 response, CDN cache status
- **Run:** `npm run verify:media-url`

2. `verify-cdn-phases.ts`

- **Purpose:** Verify CDN cache headers
- **Checks:** Cache-Control headers, immutability
- **Run:** `npm run verify:cdn-phases`

3. `verify-performance-opts.ts`

- **Purpose:** Verify performance optimizations active
- **Checks:** WebP conversion, image sizes, ISR config
- **Run:** `npm run verify:perf-opts`

11.4 Cache Warming & Maintenance

1. `warm-article-paths-cache.ts`

- **Purpose:** Pre-warm ISR cache untuk artikel populer
- **What it does:** Hit article pages untuk trigger ISR generation
- **Run:** `npm run warm:article-paths`

2. `register-ga-custom-definitions.ts`

- **Purpose:** Register GA4 custom dimensions/metrics
- **What it does:** Call GA Admin API untuk setup
- **Run:** `npm run ga:register-definitions`

12. Security

12.1 Authentication Security

JWT Validation (sudah dijelaskan di bagian 4.3):

- **Algorithm:** HS256 (HMAC SHA-256)
- **Secret:** `process.env.JWT_SECRET` (min 32 chars)
- **Expiry:** 15 minutes (access token), 30 days (refresh token)
- **Claims:** `_id, email, role, name, slug`

Token Expiry Handling:

```
// Client-side auto-refresh
export async function fetchWithAuth(url: string, options?: RequestInit) {
  let res = await fetch(url, options);

  // If 401, try refresh
  if (res.status === 401) {
    const refreshRes = await fetch("/api/auth/refresh", { method: "POST" });
    if (refreshRes.ok) {
      // Retry original request
      res = await fetch(url, options);
    } else {

```

```
// Redirect to login
window.location.href = "/api/auth/login";
}
}

return res;
}
```

Refresh Token Strategy:

- **Rotation:** Setiap refresh menghasilkan token baru, old token di-revoke
- **Single-use:** Refresh token hanya bisa dipakai sekali
- **Revocation:** Token di-revoke saat logout atau suspicious activity

Session Hijacking Prevention:

- **httpOnly Cookies:** Token tidak accessible via JavaScript
- **Secure Flag:** Cookies only sent over HTTPS
- **SameSite=Strict:** CSRF protection
- **IP Tracking:** Optional (not yet implemented)

12.2 Authorization Checks

Route Protection (sudah dijelaskan di bagian 7.1):

```
// Middleware pattern
export async function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl;

  // Protect admin routes
  if (pathname.startsWith("/admin")) {
    const user = await getUserFromRequest(request);

    if (!user) {
      return NextResponse.redirect(new URL("/api/auth/login", request.url));
    }

    const adminRoles = ["admin", "editor-in-chief", "editor", "writer"];
    if (!adminRoles.includes(user.role)) {
      return NextResponse.redirect(new URL("/", request.url));
    }
  }

  return NextResponse.next();
}

export const config = {
  matcher: ["/admin/:path*"],
};
```

API Middleware:

```
// Reusable auth middleware
export function withAuth(
  handler: (req: NextRequest, user: UserProfile) => Promise<Response>,
) {
  return async (req: NextRequest) => {
    const user = await getUserFromRequest(req);
    if (!user) {
      return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
    }
    return handler(req, user);
  };
}
```

```
}

// Usage
export const GET = withAuth(async (req, user) => {
  // user is guaranteed to exist
  const articles = await getArticlesForUser(user._id);
  return NextResponse.json({ data: articles });
});
```

Permission Validation (sudah dijelaskan di bagian 5.2):

```
// Check permission before action
if (!hasPermission(user.role, "publish_article")) {
  return NextResponse.json({ error: "Forbidden" }, { status: 403 });
}
```

Role-based Guards:

```
// Component-level guard
export function RequireRole({ children, roles }: {
  children: React.ReactNode;
  roles: string[];
}) {
  const { user } = useAuth();

  if (!user || !roles.includes(user.role)) {
    return <div>Access Denied</div>;
  }

  return <>{children}</>;
}

// Usage
<RequireRole roles={['admin', 'editor-in-chief']>
  <AdminPanel />
</RequireRole>
```

12.3 Input Validation

Zod Schema Validation:

```
// src/lib/validations/article.schema.ts
import { z } from "zod";

export const articleSchema = z.object({
  title: z
    .string()
    .min(10, "Title must be at least 10 characters")
    .max(200, "Title must be at most 200 characters"),

  excerpt: z
    .string()
    .min(50, "Excerpt must be at least 50 characters")
    .max(500, "Excerpt must be at most 500 characters"),

  content: z.string().min(100, "Content must be at least 100 characters"),

  categoryId: z.string().regex(/^([0-9a-fA-F]{24})$/, "Invalid category ID"),

  tags: z.array(z.string()).max(10, "Maximum 10 tags"),
```

```

    status: z.enum([
      "DRAFT",
      "PENDING_REVIEW",
      "PUBLISHED",
      "SCHEDULED",
      "REJECTED",
      "TAKEN_DOWN",
      "DELETED",
    ]),

    scheduledAt: z.string().datetime().optional(),
  });

// API route validation
export async function POST(request: NextRequest) {
  const body = await request.json();

  const result = articleSchema.safeParse(body);
  if (!result.success) {
    return NextResponse.json(
      { error: "Validation failed", details: result.error.issues },
      { status: 400 },
    );
  }

  // Proceed with validated data
  const article = await createArticle(result.data);
  return NextResponse.json({ data: article });
}

```

Sanitization (XSS Prevention):

```

// TipTap built-in XSS protection via schema
// HTML parsing dengan whitelist tags
import DOMPurify from "isomorphic-dompurify";

export function sanitizeHTML(html: string): string {
  return DOMPurify.sanitize(html, {
    ALLOWED_TAGS: [
      "p",
      "br",
      "strong",
      "em",
      "u",
      "a",
      "img",
      "h1",
      "h2",
      "h3",
      "u1",
      "ol",
      "li",
    ],
    ALLOWED_ATTR: ["href", "src", "alt", "class"],
  });
}

```

NoSQL Injection Prevention:

```

// Always use ObjectId untuk ID queries
import { ObjectId } from "mongodb";

// BAD: User input directly in query
const article = await collection.findOne({ _id: req.query.id }); // VULNERABLE

```



```
// GOOD: Validate & cast to ObjectId
if (!ObjectId.isValid(req.query.id)) {
  return NextResponse.json({ error: "Invalid ID" }, { status: 400 });
}
const article = await collection.findOne({ _id: new ObjectId(req.query.id) });
```

File Upload Validation:

```
export function validateUpload(file: File): { valid: boolean; error?: string } {
  const allowedTypes = ["image/jpeg", "image/png", "image/webp", "image/gif"];
  const maxSize = 10 * 1024 * 1024; // 10 MB

  if (!allowedTypes.includes(file.type)) {
    return { valid: false, error: "Invalid file type" };
  }

  if (file.size > maxSize) {
    return { valid: false, error: "File too large (max 10 MB)" };
  }

  return { valid: true };
}
```

12.4 API Security

CORS Configuration:

```
// next.config.ts
const nextConfig = {
  async headers() {
    return [
      {
        source: "/api/:path*",
        headers: [
          {
            key: "Access-Control-Allow-Origin",
            value: process.env.NEXT_PUBLIC_BASE_URL || "*",
          },
          {
            key: "Access-Control-Allow-Methods",
            value: "GET, POST, PUT, DELETE, OPTIONS",
          },
          {
            key: "Access-Control-Allow-Headers",
            value: "Content-Type, Authorization",
          },
        ],
      },
    ];
  },
};
```

Rate Limiting (conceptual, not yet implemented):

```
import { RateLimiter } from "limiter";

const limiter = new RateLimiter({
  tokensPerInterval: 100,
  interval: "minute",
});
```

```
export async function ratelimit(req: NextRequest) {
  const ip = req.ip || req.headers.get("x-forwarded-for") || "unknown";

  if (!(await limiter.removeTokens(1))) {
    return NextResponse.json({ error: "Too many requests" }, { status: 429 });
  }
}
```

Request Size Limits:

```
// next.config.ts
const nextConfig = {
  api: {
    bodyParser: {
      sizeLimit: "10mb", // Max request body size
    },
  },
};
```

Secure Headers:

```
// next.config.ts
const nextConfig = {
  async headers() {
    return [
      {
        source: "/*",
        headers: [
          { key: "X-DNS-Prefetch-Control", value: "on" },
          {
            key: "Strict-Transport-Security",
            value: "max-age=31536000; includeSubDomains",
          },
          { key: "X-Frame-Options", value: "SAMEORIGIN" },
          { key: "X-Content-Type-Options", value: "nosniff" },
          { key: "X-XSS-Protection", value: "1; mode=block" },
          { key: "Referrer-Policy", value: "strict-origin-when-cross-origin" },
        ],
      },
    ];
  },
};
```

13. Performance

13.1 Frontend Optimization

React Compiler (enabled):

```
// next.config.ts
const nextConfig = {
  experimental: {
    reactCompiler: true, // Auto-memoization
  },
};
```

Code Splitting:

```
// Automatic route-based splitting via Next.js App Router
// Manual component splitting
import dynamic from 'next/dynamic';

const TipTapEditor = dynamic(() => import('@components/tiptap/Editor'), {
  ssr: false,
  loading: () => <EditorSkeleton />,
});
```

Dynamic Imports:

```
// Load heavy libraries only when needed
const handleExport = async () => {
  const { exportToExcel } = await import("@/lib/excel-export");
  await exportToExcel(data);
};
```

Bundle Analysis:

```
npm run build
# Next.js automatic bundle analyzer
```

13.2 Image Optimization

Next.js Image Component:

```
import Image from 'next/image';

<Image
  src={article.featuredImage.url}
  alt={article.title}
  width={1200}
  height={675}
  sizes="(max-width: 768px) 100vw, (max-width: 1200px) 50vw, 33vw"
  priority={isFeatured} // LCP optimization
  placeholder="blur"
  blurDataURL={article.featuredImage.blurHash}
/>
```

Lazy Loading:

```
// Automatic via Next.js Image (default loading="lazy")
// Manual for non-Image elements
<img
  src={url}
  alt={alt}
  loading="lazy"
  decoding="async"
/>
```

Responsive Images:

```
// Served via srcset automatically by Next.js Image
// Manual for custom scenarios
<picture>
  <source srcset={` ${url}?w=320`} media="(max-width: 320px)" />
```

```
<source srcset={` ${url}?w=640`} media="(max-width: 640px)" />
<source srcset={` ${url}?w=1200`} media="(max-width: 1200px)" />
<img src={` ${url}?w=1920`} alt={alt} />
</picture>
```

WebP Format: Sudah dijelaskan di bagian 8.2

13.3 Caching Strategy

Browser Cache Headers (dari S3):

```
Cache-Control: public, max-age=31536000, immutable
```

CDN Caching:

- **Static Assets:** 1 year cache
- **HTML Pages:** ISR revalidation based
- **API Responses:** No cache (dynamic)

Database Query Caching (Not implemented, future):

```
// Redis cache layer
import { Redis } from "ioredis";

const redis = new Redis(process.env.REDIS_URL);

export async function getCachedArticle(id: string) {
  const cached = await redis.get(`article:${id}`);
  if (cached) return JSON.parse(cached);

  const article = await getArticleById(id);
  await redis.set(`article:${id}`, JSON.stringify(article), "EX", 3600); // 1 hour
  return article;
}
```

API Response Caching (ISR for Server Components):

```
// src/app/(public)/[category]/[yyyy]/[mm]/[dd]/[slug]/page.tsx
export const revalidate = 3600; // 1 hour
```

13.4 Database Performance

Index Optimization (sudah dijelaskan di bagian 3):

- Compound indexes untuk common queries
- Text indexes untuk search
- Sparse indexes untuk optional fields

Query Optimization:

```
// Use projection untuk exclude heavy fields
db.articles.find(
  { status: "PUBLISHED" },
  { projection: { content: 0, revisionHistory: 0 } },
);

// Use limit untuk pagination
db.articles.find().limit(10);
```

```
// Use covered queries (query + sort + projection all covered by index)
db.articles
  .find({ status: "PUBLISHED" }, { projection: { _id: 1, title: 1 } })
  .sort({ publishedAt: -1 });
```

Connection Pooling: Sudah dijelaskan di bagian 6.1

Read Replicas: Not yet implemented (future scaling)

14. Monitoring & Observability

14.1 Logging

Pino Logger Configuration:

```
// src/lib/logger.ts
import pino from "pino";

export const logger = pino({
  level: process.env.LOG_LEVEL || "info",
  formatters: {
    level: (label) => ({ level: label }),
  },
  ...(process.env.NODE_ENV === "development" && {
    transport: {
      target: "pino-pretty",
      options: {
        colorize: true,
        translateTime: "SYS:standard",
        ignore: "pid,hostname",
      },
    },
  }),
});

// Usage
logger.info({ articleId, userId }, "Article published");
logger.error({ error, context }, "Failed to upload media");
```

Log Levels:

- **fatal:** Application crash
- **error:** Error yang perlu immediate attention
- **warn:** Warning (tidak critical)
- **info:** General info (default)
- **debug:** Debug info (development)
- **trace:** Very verbose (development)

Structured Logging:

```
logger.info(
  {
    event: "article_published",
    articleId: article._id,
    authorId: article.authorId,
    category: article.category.slug,
    timestamp: new Date().toISOString(),
  },
  "Article published successfully",
);
```

Log Aggregation: Not yet implemented (future: CloudWatch, Datadog, Sentry)

14.2 Error Tracking

Error Boundaries (React):

```
// src/components/ErrorBoundary.tsx
'use client';

import { Component, type ReactNode } from 'react';

export class ErrorBoundary extends Component<
  { children: ReactNode },
  { hasError: boolean }
> {
  constructor(props: any) {
    super(props);
    this.state = { hasError: false };
  }

  static getDerivedStateFromError() {
    return { hasError: true };
  }

  componentDidCatch(error: Error, errorInfo: any) {
    logger.error({ error, errorInfo }, 'React error boundary caught');
    // Send to error tracking service
  }

  render() {
    if (this.state.hasError) {
      return <div>Something went wrong. Please refresh.</div>;
    }
    return this.props.children;
  }
}
```

Error Reporting: Not yet implemented (future: Sentry)

Stack Traces: Automatic via logger

User Context: Include `userId`, `path`, `userAgent` dalam error logs

14.3 Performance Monitoring

Core Web Vitals (via GA4):

- **LCP** (Largest Contentful Paint): < 2.5s
- **FID** (First Input Delay): < 100ms
- **CLS** (Cumulative Layout Shift): < 0.1

API Response Times: Log via Pino

Database Query Times: Monitor via MongoDB Atlas

Resource Utilization: Monitor via hosting platform (Vercel, etc)

15. Development Workflow

15.1 Environment Setup

Prerequisites:

- Node.js 20+

- MongoDB 7.1+
- AWS S3 (atau MinIO untuk local)
- Firebase project

Installation:

```
git clone <repo>
cd arasvara
npm install
cp .env.example .env.local
# Edit .env.local dengan credentials
npm run dev
```

Environment Variables: Lihat section 17.2 (Appendix)

Database Seeding:

```
tsx scripts/seed.ts
```

15.2 Local Development

Dev Server:

```
npm run dev
# Open http://localhost:3000
```

Hot Reload: Automatic via Next.js Fast Refresh

Database Connection: Local MongoDB atau MongoDB Atlas

External Services Mocking: MinIO untuk S3, Firebase Emulator (optional)

15.3 Testing

Unit Tests (Vitest):

```
npm run test          # Run once
npm run test:watch     # Watch mode
```

Test Coverage: Not yet comprehensive

Test Data Fixtures: [src/lib/db/seed.ts](#)

15.4 Build & Deployment

Build Process:

```
npm run build
npm start
```

Environment Promotion:

- **dev:** Local development
- **staging:** Pre-production testing
- **prod:** Production

Deployment Checklist:

1. Run tests
2. Build locally
3. Check env vars
4. Deploy
5. Smoke test
6. Monitor logs

Rollback Procedure: Revert deployment via hosting platform

16. Troubleshooting

16.1 Common Issues

Authentication Failures:

- Check JWT_SECRET set
- Check cookie domain
- Check Firebase credentials

Database Connection Issues:

- Check MONGO_URL
- Check IP whitelist (MongoDB Atlas)
- Check connection pool exhausted

S3 Upload Failures:

- Check S3 credentials
- Check bucket permissions
- Check presigned URL expiry

Image Processing Errors:

- Check Sharp installation (native module)
 - Check file format supported
 - Check memory limits
-

16.2 Debug Tips

Logging Techniques: Use `logger.debug()` liberally

Browser DevTools: Network tab untuk API calls, Console untuk errors

Database Query Debugging: Use MongoDB Compass atau Atlas Query Profiler

Network Inspection: Check request/response headers, payload

16.3 Known Limitations

File Size Limits:

- Images: 10 MB
- Videos: 100 MB

Rate Limits: Not yet implemented

Browser Compatibility: Modern browsers only (ES2020+)

17. Appendix

17.1 Type Definitions Reference

Lihat [src/types/](#) untuk semua type definitions.

Key types:

- [Article](#), [ArticleStatus](#), [ArticleFormData](#)
 - [User](#), [UserProfile](#), [Role](#)
 - [Category](#), [CategoryWithParent](#)
 - [Media](#), [PendingMedia](#)
 - [Notification](#), [NotificationType](#)
-

17.2 Environment Variables Reference

Database:

- [MONGO_URL](#) - MongoDB connection string
- [DB_NAME](#) - Database name (default: arasvara_news)

Auth:

- [JWT_SECRET](#) - JWT signing secret (min 32 chars)

S3/MinIO:

- [S3_REGION](#), [S3_ENDPOINT](#), [S3_ACCESS_KEY](#), [S3_SECRET_KEY](#)
- [S3_BUCKET_NAME](#), [S3_BUCKET_AVATAR](#), [S3_BUCKET_CONFIGURATION](#)
- [S3_FORCE_PATH_STYLE](#), [S3_MAX_SOCKETS](#)

Next.js Public:

- [NEXT_PUBLIC_BASE_URL](#), [NEXT_PUBLIC_API_URL](#)
- [NEXT_PUBLIC_STORAGE_MEDIA](#), [NEXT_PUBLIC_STORAGE_CONFIGURATION](#)
- [NEXT_PUBLIC_ADMIN_PANEL_PATH](#)

Firebase:

- [FIREBASE_SERVICE_ACCOUNT](#) (JSON string)
- [NEXT_PUBLIC_FIREBASE_API_KEY](#), [NEXT_PUBLIC_FIREBASE_PROJECT_ID](#), [dll](#)

Google Analytics:

- [NEXT_PUBLIC_GTM_ID](#), [NEXT_PUBLIC_GA_MEASUREMENT_ID](#)
- [GA_MP_API_SECRET](#)

Feature Flags:

- [ARTICLE_STRUCTURED_URL_ENABLED](#) (default: true)
 - [ARTICLE_PAGE_REVALIDATE_SECONDS](#) (default: 3600)
-

17.3 Database Indexes Reference

Lihat bagian 3 (Database Schema) untuk detail indexes per collection.

17.4 External API Documentation Links

- **MongoDB:** <https://www.mongodb.com/docs/>
 - **AWS S3:** <https://docs.aws.amazon.com/s3/>
 - **Firebase:** <https://firebase.google.com/docs>
 - **Google Analytics 4:** <https://developers.google.com/analytics/devguides/collection/ga4>
 - **Next.js:** <https://nextjs.org/docs>
 - **TipTap:** <https://tiptap.dev/docs>
 - **Sharp:** <https://sharp.pixelplumbing.com/>
 - **Pino:** <https://getpino.io/>
-

17.5 Glossary

- **ISR**: Incremental Static Regeneration
- **SSR**: Server-Side Rendering
- **SSG**: Static Site Generation
- **CDN**: Content Delivery Network
- **RBAC**: Role-Based Access Control
- **JWT**: JSON Web Token
- **FCM**: Firebase Cloud Messaging
- **GA4**: Google Analytics 4
- **ULID**: Universally Unique Lexicographically Sortable Identifier
- **WebP**: Modern image format dengan kompresi superior
- **TipTap**: Headless WYSIWYG editor
- **Pino**: Fast JSON logger untuk Node.js
- **Sharp**: High-performance image processing library
- **Zod**: TypeScript-first schema validation

End of Documentation

Last Updated: 2026-07-30

Version: 1.0

Maintainer: Arasvara Tech Team